

Backend khả vi và hệ VO/SLAM đầu-cuối (Differentiable Backends and End-to-End Systems)

PHỤ THUỘC F/27-dat-hoc-sau-vao-dau (khung bốn cổng để quyết định có nên thay một khối); A/01-toan-toi-thieu/02-vi-phan-va-toi-uu (Gauss-Newton, Levenberg-Marquardt, quy tắc chuỗi); C/15-thi-giac-khong-thoi-gian/03-flow-tracking-va-rang-buoc (dòng quang và toán tử cập nhật hồi quy); F/28-dac-trung-va-ghep-cap là nửa frontend của cùng câu chuyện.

TÓM TẮT

Bạn đã thạo Gauss-Newton, Levenberg-Marquardt và đồ thị nhân tử. Chương này không dạy lại chúng. Nó nói về đúng một thứ mà chữ “khả vi” thêm vào: cho *gradient* chạy *xuyên qua* lời giải của bài toán tối ưu, để những con số bạn đang đặt bằng tay được huấn luyện theo lỗi quỹ đạo cuối cùng. Đọc xong bạn viết được đạo hàm của một nghiệm arg min bằng định lý hàm ẩn, viết ra được công thức chi phí bộ nhớ của lượt ngược qua một lớp tối ưu, và đọc DROID-SLAM hay CodeSLAM mà biết ngay chỗ nào là hình học cũ, chỗ nào là phần học được. Nếu chỉ nhớ một điều: khả vi *không* làm bộ giải tốt lên, nó chỉ biến các nút vặn thủ công thành tham số học được — và cái giá là bộ nhớ lượt ngược, độ ổn định số ở đúng những khung hình khó, cùng gần hết khả năng chẩn đoán khi hệ hỏng.

KIẾN THỨC CẦN CÓ TRƯỚC

- **bài toán bình phương tối thiểu phi tuyến** — tìm x làm nhỏ nhất tổng bình phương của các phần dư $r_i(x)$. Trong SLAM, x là tư thế và điểm bản đồ, còn r_i là sai lệch chiếu lại của một quan sát.
- **ma trận thông tin (information matrix)** — nghịch đảo hiệp phương sai của một phép đo, và cũng là trọng số của cạnh đó trong bài toán bình phương tối thiểu. Đặt lớn nghĩa là “tôi tin phép đo này”.
- **Hessian** H — ma trận hệ phương trình chuẩn của Gauss-Newton, $H = J^T \Sigma^{-1} J$. Nghịch đảo của nó xấp xỉ hiệp phương sai của nghiệm, tức thứ bạn dùng khi biên hoá một khung hình.
- **tự do đo (gauge freedom)** — các hướng mà bài toán không xác định được. Dịch cả quỹ đạo đi một khoảng thì lỗi không đổi. Theo những hướng đó H suy biến, nên phải neo gốc toạ độ hoặc thêm damping.
- **damping** λ — số cộng vào đường chéo của H trong Levenberg-Marquardt. Nó vừa làm hệ khả nghịch, vừa rút ngắn bước đi khi mô hình tuyến tính hoá không đáng tin.
- **đạo hàm tự động (autodiff)** — kỹ thuật để máy tính đạo hàm của một chương trình bằng cách áp quy tắc chuỗi lên từng phép toán. **lan truyền ngược (backpropagation)** là autodiff chạy theo chiều ngược.
- **lượt xuôi và lượt ngược** — lượt xuôi là một lần chạy mạng để ra kết quả; lượt ngược là một lần đi ngược để tính đạo hàm. Lượt ngược cần các giá trị trung gian của lượt xuôi, nên phải giữ chúng trong bộ nhớ.
- **tự giám sát (self-supervised)** — huấn luyện không cần nhãn do người tạo. Tín hiệu dạy đến từ chính dữ liệu, ví dụ “dừng lại ảnh này từ ảnh kia thì phải khớp”.

CÂU HỎI MỞ ĐẦU

- Q1. Backend của bạn đã tối ưu tốt rồi. Vậy chữ “khả vi” còn thêm được gì, và nếu nó không làm Gauss-Newton hội tụ nhanh hơn thì nó bán cái gì?
- Q2. Đạo hàm của một nghiệm arg min theo tham số đầu vào tính được mà không cần lưu bước lặp nào. Vì sao lại có chuyện đó, và điều kiện nào phải đúng để nó không sai?
- Q3. Quyết định inlier hay outlier là một phép so sánh với ngưỡng, nên đạo hàm của nó bằng 0 gần như ở mọi nơi. Vậy một hệ đầu-cuối học cách loại điểm sai bằng cách nào?
- Q4. DROID-SLAM chính xác hơn hẳn nhiều hệ cổ điển trên *benchmark* công khai. Vì sao nó vẫn không phải lựa chọn cho một robot chạy Jetson Orin, và cái gì chặn nó?
- Q5. Nén cả bản đồ độ sâu vào 32 con số rồi tối ưu chung với tư thế nghe rất gọn. Ý tưởng đó ra đời năm 2018 và tới nay vẫn chưa thắng. Nó hỏng ở chỗ nào?

1.1 Đặt vấn đề

Backend của bạn không hỏng. Nó tuyến tính hoá, nó dùng phần bù Schur, nó chạy nhân Huber, nó biên hoá khung hình cũ. Đưa cho nó các phần dư đúng và các trọng số đúng thì nó trả về nghiệm tốt nhất theo nghĩa bình phương tối thiểu. Vấn đề nằm ở chỗ khác: **ai quyết định các trọng số đó.**

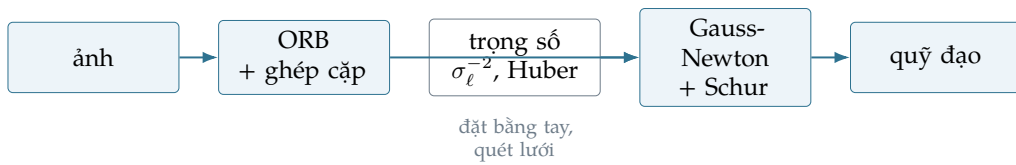
Hãy liệt kê thật cụ thể, trong một hệ kiểu ORB-SLAM3 [1]:

- **Trọng số cạnh chiếu lại** — ma trận thông tin là $\sigma_\ell^{-2}I$, với σ_ℓ suy từ hệ số tỉ lệ của kim tự tháp ORB. Nó chỉ phụ thuộc vào *tầng* mà điểm được phát hiện. Nó không biết mảnh ảnh đó có bị mờ không, có nằm trên vệt loá không, có nằm trên một hoa văn lặp không.
- **Bề rộng nhân robust** — ngưỡng χ^2 5,991 cho cạnh hai bậc tự do và 7,815 cho cạnh ba bậc. Đây là phân vị 95% của một giả định phân phối chuẩn, và giả định ấy hiếm khi được kiểm lại trên dữ liệu thật.
- **Nhiều IMU** — lấy từ tờ thông số cảm biến, cộng thêm một bước ngẫu nhiên cho độ trôi. Cùng một con số cho lúc đứng yên và lúc quay đầu 200 độ mỗi giây.
- **Các ngưỡng rời rạc** — tỉ lệ khoảng cách mô tả, ngưỡng inlier của RANSAC, chính sách chèn khung then chốt.

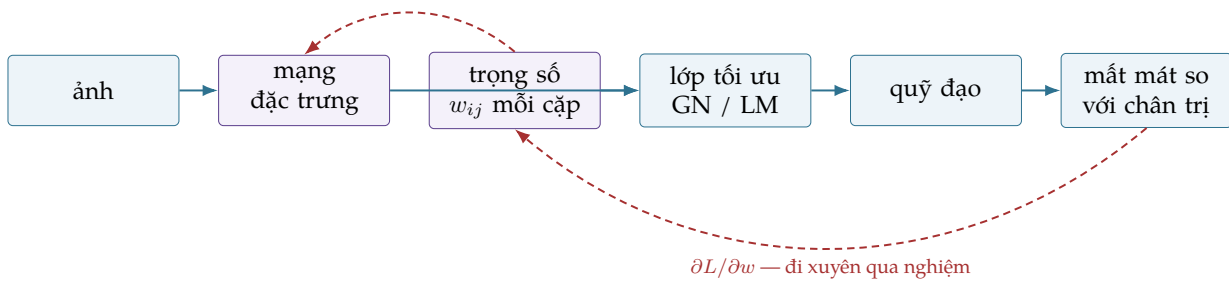
Mọi con số trên đều trả lời cùng một câu hỏi: *tôi nên tin phép đo này bao nhiêu*. Và mọi con số trên đều được đặt một lần, toàn cục, từ một tờ thông số hoặc một bài báo. Trong khi đó bạn có một đại lượng thật sự quan tâm ở cuối đường ống, là sai số quỹ đạo tuyệt đối. Giữa các nút vặn và đại lượng đó **không có đạo hàm nào**. Nên bạn đóng khoảng trống ấy bằng cách duy nhất còn lại: quét lưới, chạy lặp nhiều lần, đọc bảng. Đó chính là lý do tồn tại của mọi *benchmark* harness.

Chữ **khả vi** (*differentiable*) trong chương này là lời hứa rằng đạo hàm ấy tồn tại được. Nếu toàn bộ đường đi từ điểm ảnh tới quỹ đạo là một đồ thị phép tính khả vi, thì đạo hàm của APE theo trọng số cạnh là một đại lượng tính được. Khi đó các trọng số trở thành tham số học được. Hình 1.1 vẽ đúng sự khác biệt: mũi tên xuôi giống nhau ở hai phía, mũi tên ngược chỉ tồn tại ở phía dưới.

Hệ cổ điển



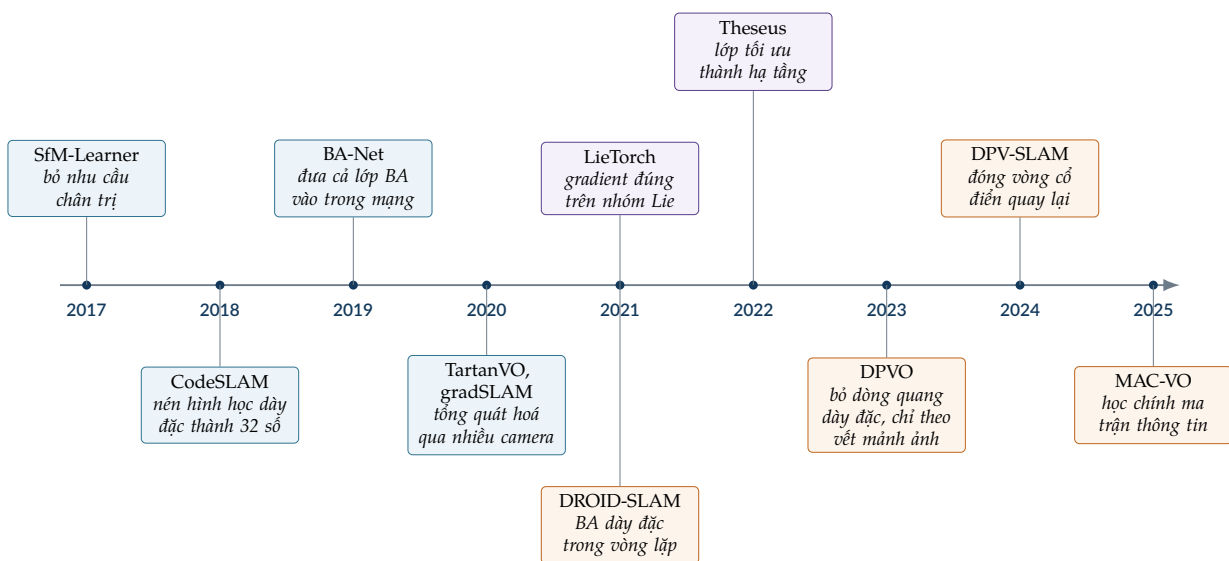
Hệ khả vi



Hình 1.1. Cái mà chữ “khả vi” thêm vào. Chiều xuôi, vẽ bằng mũi tên liền, giống hệt nhau ở hai hệ: đặc trưng, trọng số, bộ giải, quỹ đạo. Khác biệt duy nhất là mũi tên đứt màu đỏ. Nó đi ngược *xuyên qua* lời giải của bài toán tối ưu, nhờ đó các trọng số w_{ij} và mạng sinh ra chúng được huấn luyện theo lỗi quỹ đạo cuối cùng, thay vì được đặt bằng tay từ dữ liệu thông số cảm biến.

Nói gọn giá trị của chương: học xong bạn phân biệt được ba nhóm hay bị gộp làm một. Nhóm thứ nhất là *lớp tối ưu khả vi*, một mảnh hạ tầng có thể lấy riêng. Nhóm thứ hai là *hệ đầu-cuối* kiểu DROID-SLAM, một kiến trúc trọn gói và đắt. Nhóm thứ ba là *VO tự giám sát*, vốn là một cách huấn luyện chứ không phải một hệ SLAM. Ba thứ đó có chi phí và rủi ro rất khác nhau. Trộn chúng lại là cách nhanh nhất để đánh giá sai cả ba.

Hình 1.2 xếp các công trình của chương lên một trục thời gian, và ghi cạnh mỗi mốc *nút thắt mà nó gỡ*. Đọc hình đó trước sẽ nhanh hơn đọc tuần tự, vì cả chương chỉ là một chuỗi lời đáp cho những nút thắt liên tiếp nhau.



Hình 1.2. Dòng thời gian của chương, mỗi mốc ghi kèm nút thắt mà nó gỡ. Hộp tím là hạ tầng, tức thứ dùng lại được ở mọi hệ. Hộp cam là các hệ đầu-cuối, và ba mốc cuối cho thấy hướng đi chung của chúng: bớt dày đặc, mượn lại đóng vòng cổ điển, rồi chuyển mục tiêu học từ đặc trưng sang ma trận thông tin — tức là càng ngày càng gần với một backend cổ điển được cấp trọng số tốt hơn.

1.2 Đạo hàm xuyên qua một nghiệm tối ưu (Differentiating Through an Argmin)

A. Bài toán phát biểu chính xác

Gọi θ là mọi thứ mạng sinh ra: trọng số cạnh, mục tiêu dòng quang, hệ số damping. Gọi

$$x^*(\theta) = \arg \min_x E(x, \theta)$$

là nghiệm mà bộ giải trả về, với x là tư thế và hình học. Ở cuối có một hàm mất mát $L(x^*(\theta))$, thường là sai lệch so với quỹ đạo chân trị. Ta cần $\partial L / \partial \theta$.

Chỗ khó là x^* không có công thức đóng. Nó là kết quả của một vòng lặp. Có đúng hai cách lấy đạo hàm qua nó, và hai cách đánh đổi ngược nhau.

B. Cách thứ nhất: trải phẳng vòng lặp

Cách thô là coi K bước lặp như một mạng K tầng rồi để autodiff làm việc:

$$x_{k+1} = x_k - (H_k + \lambda I)^{-1} g_k.$$

Mỗi bước là một biểu thức khả vi, nên cả vòng lặp cũng vậy. Cách này gọi là **trải phẳng** (*unrolling*).

Ưu điểm của nó là tính trung thực. Đạo hàm thu được là đạo hàm của *thuật toán thật sự đã chạy*, kể cả khi thuật toán đó mới chạy năm bước và chưa hội tụ. Nhược điểm là bộ nhớ. Lướt ngược cần mọi giá trị trung gian của lượt xuôi. Bạn phải giữ cả K bộ Jacobian, K bản phân tích Cholesky, và mọi bản đồ đặc trưng đã sinh ra chúng.

Chi phí ấy nên nhớ dưới dạng công thức, vì công thức mới là thứ dùng lại được. Gọi K là số bước lặp, N là số khung trong cửa sổ, C là số kênh của bản đồ đặc trưng, và HW là số vị trí ở độ phân giải đang chạy. Bộ nhớ lướt ngược tỉ lệ với tích $K N C H W$ — tuyến tính theo *cả bốn* thừa số cùng lúc, và một bước lặp hồi quy còn giữ vài tensor cùng cỡ nên hằng số nhân phía trước cũng không nhỏ. Đó là lý do các hệ trải phẳng thật đều huấn luyện với cửa sổ ngắn và độ phân giải thấp: N và HW là hai thừa số duy nhất người dùng cắt được mà không phải đổi kiến trúc. Nếu bạn muốn một con số gigabyte cho hệ của mình thì phải tự đo; công thức chỉ nói hình dạng, không nói độ lớn.

C. Cách thứ hai: định lý hàm ẩn

Cách thứ hai không lưu bước lặp nào. Nó dùng đúng một sự kiện: tại nghiệm, **gradient** (*gradient*) theo x bằng không.

$$g(x^*, \theta) = \left. \frac{\partial E}{\partial x} \right|_{x^*} = 0.$$

Đây là một hệ phương trình ràng buộc x^* với θ . Lấy vi phân toàn phần hai vế theo θ :

$$\underbrace{\frac{\partial^2 E}{\partial x^2}}_H \frac{\partial x^*}{\partial \theta} + \underbrace{\frac{\partial^2 E}{\partial x \partial \theta}}_B = 0 \quad \implies \quad \frac{\partial x^*}{\partial \theta} = -H^{-1} B.$$

Đây là **định lý hàm ẩn** (*implicit function theorem*) áp cho điều kiện tối ưu. Nó nói rằng nghiệm trượt đi theo hướng nào khi ta đẩy tham số, và câu trả lời chỉ phụ thuộc hai thứ: độ cong H tại nghiệm, và mức độ tham số đẩy vào gradient qua B .

Với hàm mất mát cuối, ta không cần cả ma trận $\partial x^* / \partial \theta$. Chỉ cần một tích vectơ:

$$\frac{\partial L}{\partial \theta} = - \left(\frac{\partial L}{\partial x} \right)^\top H^{-1} B = -v^\top B, \quad \text{với } H v = \frac{\partial L}{\partial x}.$$

Đọc dòng cuối cho kỹ, vì đó là điểm mấu chốt của cả chương. **Lượt ngược chỉ tốn thêm đúng một lần giải hệ, với chính ma trận H mà lượt xuôi đã phân tích rồi.** Với cửa sổ 8 khung, hệ camera rút gọn chỉ là 48×48 , tức nhỏ hơn nhiều bậc so với chính lượt xuôi vừa chạy, nên lượt ngược không thêm khoản chi phí nào đáng kể. Bộ nhớ cho vòng lặp bằng không, vì không bước lặp nào được lưu.

Còn một cách đọc nữa mà kỹ sư SLAM sẽ thấy quen ngay. H^{-1} chính là hiệp phương sai của nghiệm, đúng đại lượng bạn đã tính khi biên hoá một khung hình. Cổ máy cần cho lượt ngược đã nằm sẵn trong mã nguồn của bạn. Ý tưởng lấy đạo hàm qua một bài toán tối ưu không sinh ra từ thị giác; nó đến từ dòng lớp tối ưu khả vi trong học máy [2], [3], và cùng một thủ thuật cũng là thứ làm cho mô hình cân bằng sâu chạy được [4].

TRỰC GIÁC

Hãy hình dung nghiệm là một hòn bi nằm dưới đáy một cái bát. Bạn không quan tâm nó lăn xuống đáy bằng đường nào. Bạn chỉ quan tâm một việc: nghiêng cái bát đi một chút thì hòn bi dịch bao nhiêu. Trả lời câu đó chỉ cần biết *độ cong của cái bát tại đáy* và *lực nghiêng đáy vào đâu*. Đó đúng là H^{-1} và B . Cách trải phẳng thì ngược lại: nó quay phim toàn bộ đường lăn rồi tua ngược. Tốn băng — nhưng nếu hòn bi chưa lăn tới đáy thì đoạn phim vẫn dùng được, còn cái bát thì không.

D. Ví dụ tính tay được

Lấy bài toán nhỏ nhất còn giữ nguyên bản chất: một ẩn, hai phép đo, một trọng số.

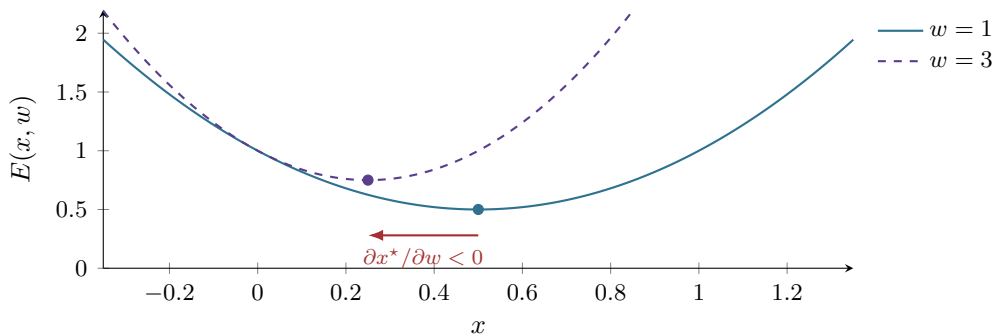
$$E(x, w) = w(x - a)^2 + (x - b)^2.$$

Nghiệm có công thức đóng, $x^* = (wa + b)/(w + 1)$. Bây giờ tính lại bằng định lý hàm ẩn. Ta có $H = 2(w + 1)$ và $B = \partial^2 E / \partial x \partial w = 2(x - a)$, nên

$$\frac{\partial x^*}{\partial w} = -\frac{2(x^* - a)}{2(w + 1)} = -\frac{x^* - a}{w + 1}.$$

Thay $a = 0, b = 1, w = 1$. Khi đó $x^* = 0,5$ và đạo hàm bằng $-0,25$. Kiểm tra trực tiếp: $x^* = 1/(w + 1)$, đạo hàm là $-1/(w + 1)^2 = -1/4$. Khớp.

Con số $-0,25$ nói điều gì bằng lời thường? Tăng trọng số của phép đo a lên 0,1 thì nghiệm dịch về phía a khoảng 0,025. Nếu hàm mất mát cuối muốn nghiệm nhỏ đi, **gradient** sẽ bảo mạng “hãy tin phép đo a hơn”. Đó chính xác là thứ một hệ đầu-cuối học, chỉ khác là với hàng chục nghìn phép đo cùng lúc. Hình 1.3 vẽ lại cùng ý đó bằng hình học.



Hình 1.3. Định lý hàm ẩn nhìn bằng mắt, cho $E = wx^2 + (x - 1)^2$. Tăng trọng số w của phép đo tại $x = 0$ làm đường cong dốc hơn về phía trái, nên đáy trượt từ $x^* = 0,5$ sang $x^* = 0,25$. Đạo hàm $-H^{-1}B$ chính là độ dốc của đường trượt ấy. Nó chỉ cần độ cong tại đáy, chứ không cần biết bộ giải đã đi đường nào tới đó.

E. Ba điều kiện, và cả ba đều bị vi phạm trong SLAM

Định lý hàm ẩn đẹp nhưng có điều kiện. Nó cần x^* là điểm dừng thật, cần H khả nghịch, và cần E đủ trơn quanh nghiệm. Trong SLAM cả ba đều lung lay.

- **H suy biến theo cấu tạo.** Bài toán đơn ảnh có bảy hướng tự do đo. Ngay cả stereo-inertial vẫn còn vài hướng nếu chưa neo gốc. Theo những hướng đó H không khả nghịch. Cách chữa duy nhất là neo gốc hoặc cộng damping, và *giá trị damping đổi thì đạo hàm cũng đổi*. Bạn đang huấn luyện theo một **gradient** phụ thuộc vào một lựa chọn số học, không phải vào vật lý.
- **Bộ giải bị cắt ngắn.** Năm bước LM từ một khởi tạo tồi không cho một điểm dừng. Khi đó định lý hàm ẩn trả về đạo hàm của một bài toán mà bạn *chưa* giải. Sai lệch này hiếm khi được đo, và nó lớn nhất đúng ở những khung khó.
- **Có những quyết định không khả vi.** Đây là điều sâu nhất trong chương. So sánh χ^2 với ngưỡng, chọn inlier trong RANSAC, quyết định chèn khung then chốt — tất cả đều là hàm bậc thang. Đạo hàm của chúng bằng 0 hầu khắp nơi và không tồn tại tại chỗ nhảy. **Gradient** không nhìn thấy chúng.

Gạch đầu dòng cuối giải thích được toàn bộ thiết kế của các hệ trong chương này. Vì **không thể học một quyết định cứng**, mọi hệ đầu-cuối đều thay quyết định cứng bằng một *trọng số mềm liên tục*. DROID-SLAM không xuất ra mặt nạ inlier; nó xuất ra một con số tin cậy cho từng điểm ảnh. MAC-VO không xuất ra nhãn “điểm này tốt”; nó xuất ra một hiệp phương sai. Đó không phải một lựa chọn thẩm mỹ mà là hệ quả bắt buộc của việc muốn có đạo hàm.

→ cùng nguyên lý với B/09-thiet-ke-ham-mat-mat — lại là quy tắc “muốn học được thì phải làm cho trơn”, cùng lý do focal loss thay cho việc chọn cứng mẫu khó

F. Vì sao gradient nổ đúng ở khung khó — một phép tính EuRoC

Điều kiện “ H khả nghịch” nghe như một chi tiết kỹ thuật. Thực ra nó là chế độ hỏng nặng nhất trong huấn luyện, và nó tính ra được bằng số liệu bạn đã quen.

Xét một điểm bản đồ quan sát bằng stereo. Bỏ qua các hướng khác, giữ hai bậc tự do: dịch ngang và độ sâu. Với tiêu cự f và đường cơ sở b , độ bất định của hai hướng là

$$\sigma_x = \frac{Z}{f} \sigma_u, \quad \sigma_Z = \frac{Z^2}{fb} \sigma_d,$$

với σ_u là sai số định vị điểm ảnh và σ_d là sai số disparity. Lấy $\sigma_d = \sigma_u$ cho gọn, tỉ số rút lại còn một biểu thức rất sạch:

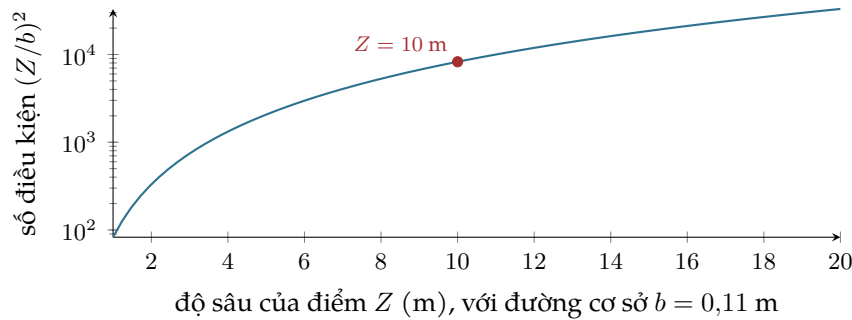
$$\frac{\sigma_Z}{\sigma_x} = \frac{Z}{b}.$$

Bây giờ thay số vào công thức — đây là phép thay số, không phải phép đo trên máy nào. Lấy thông số công khai của EuRoC, $b = 0,11$ m và $f \approx 460$ px, rồi *giả sử* $\sigma_u = 0,5$ px và một điểm ở $Z = 10$ m. Khi đó $\sigma_Z/\sigma_x = Z/b$ cho tỉ số cỡ chín chục lần: bất định theo hướng độ sâu lớn hơn bất định theo hướng ngang gần hai bậc độ lớn.

Ma trận thông tin là nghịch đảo bình phương của các độ lệch đó, nên khối 2×2 của H có số điều kiện khoảng $(Z/b)^2 \approx 8300$. Đây là con số quyết định. Trong công thức $v = H^{-1} \partial L / \partial x$, thành phần **gradient** theo hướng độ sâu bị nhân lên khoảng tám nghìn lần so với thành phần theo hướng ngang. Nói bằng lời: **ước lượng càng ít ràng buộc một hướng, thì đạo hàm theo hướng đó càng lớn**, vì một thay đổi nhỏ ở trọng số làm nghiệm trượt rất xa theo hướng yếu.

Hệ quả cho huấn luyện thì cụ thể và khó chịu. Các khung thị sai thấp, chuyển động gần như quay thuần túy, và ảnh mờ đều làm Z/b hiệu dụng lớn lên. Đó lại đúng là những khung bạn muốn mạng học từ chúng. Nên **gradient** từ mẫu quý nhất cũng là **gradient** nhiều nhất và dễ nổ

nhất. Mọi cách chữa thông dụng — cắt chuẩn *gradient*, tăng damping, bỏ qua mẫu có H xấu — đều làm giảm ảnh hưởng của đúng nhóm mẫu ấy. Hình 1.4 cho thấy số điều kiện tăng theo bình phương độ sâu.



Hình 1.4. Số điều kiện của khối hai bậc tự do (ngang, sâu) cho một điểm stereo trên cấu hình EuRoC. Đây là đường của công thức $(Z/b)^2$, không phải một tập điểm đo. Nó chính là hệ số khuếch đại mà H^{-1} áp lên *gradient* theo hướng độ sâu trong lượt ngược. Điều đáng nhớ là hình dạng: hệ số tăng theo bình phương độ sâu, nên tăng độ sâu gấp đôi thì nó tăng gấp bốn. Đây là lý do định lượng khiến huấn luyện qua một lớp tối ưu mất ổn định trên đúng những khung ít thị sai.

G. Khối nào trong hệ của bạn học được, khối nào không

Gộp hai kết luận vừa rồi — *gradient* chỉ chảy qua thứ liên tục, và nó nổ ở chỗ H xấu — ta có một bản kiểm kê rất cụ thể cho một hệ kiểu ORB-SLAM3. Bảng 1.1 là bảng đáng dán lên tường trước khi bắt đầu bất cứ thí nghiệm nào trong chương này.

Bảng 1.1. Kiểm kê các khối của một hệ stereo-inertial cổ điển theo tiêu chí “*gradient* có chảy qua được không”. Hai dòng in đậm là hai chỗ rẻ nhất để bắt đầu, vì chúng đã liên tục sẵn và không đòi thay kiến trúc.

Khối	Đầu ra	Liên tục?	Học được thể nào
Trích đặc trưng ORB	vị trí + mô tả nhị phân	không	Phải thay bằng bản đồ nhiệt và mô tả số thực; đó là chương 28
Ghép cặp + kiểm tra tỉ lệ	tập cặp khớp	không	Thay bằng ma trận gán mềm rồi mới lấy <i>gradient</i> qua nó
RANSAC và cổng χ^2	mặt nạ inlier	không	Thay hẳn bằng trọng số liên tục; không có cách nào khác
Ma trận thông tin cạnh	số thực dương	có	Học trực tiếp; đây là điểm vào rẻ nhất của cả chương
Bề rộng nhân robust	số thực dương	có	Học được, nhưng <i>gradient</i> yếu vì nhân robust bão hoà ở phần đuôi
Hiệu chỉnh chùm tia	nghiệm	có	Đi xuyên qua bằng định lý hàm ẩn, với điều kiện đã neo gốc
Chính sách chèn khung then chốt	quyết định đúng/sai	không	Không; phải giữ luật viết tay hoặc học bằng cách khác hẳn
Lấy ứng viên đóng vòng	xếp hạng + ngưỡng	một nửa	Học được phần <i>embedding</i> , không học được phần ngưỡng; chương 29
Kiểm chứng Sim3 + RANSAC	quyết định đúng/sai	không	Không
Tối ưu đồ thị tư thế	nghiệm	có	Đi xuyên qua được, nhưng đồ thị lớn nên lượt ngược đắt
Khởi tạo IMU	nghiệm + cổng chấp nhận	một nửa	Chỉ phần nghiệm; cổng chấp nhận vẫn rời rạc

Bảng này trả lời luôn câu hỏi “đầu-cuối tới mức nào”. Sáu trên mười một khối không cho *gradient* đi qua, và trong sáu khối đó có cả đóng vòng lẫn chèn khung then chốt — hai thứ chi phối sai số dài hạn của bạn. Nên không có hệ nào trong chương thật sự đầu-cuối theo nghĩa đen. Chúng chỉ đầu-cuối trên phần vốn đã trơn.

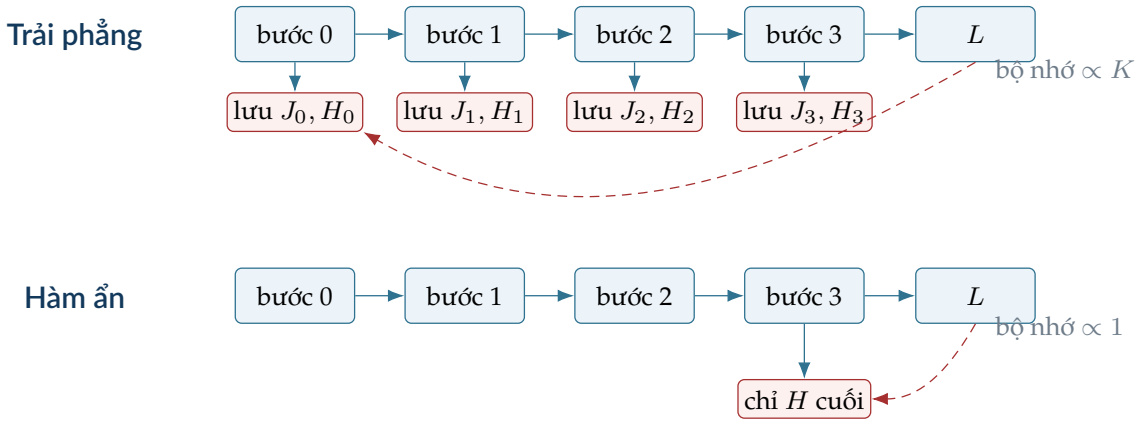
H. Ba cái giá, nói thẳng

Bảng 1.2 tổng kết cái được và cái mất. Cột cuối là cột đáng đọc nhất.

Bảng 1.2. Khả vi mua được gì và trả bằng gì. Cột “hỏng khi nào” là cột quyết định: nó cho biết trong hoàn cảnh nào một hệ khả vi trở nên tệ hơn hệ cổ điển, chứ không chỉ đắt hơn.

Chiều	Được gì	Trả bằng gì	Hỏng khi nào
Trọng số đo lường	Trọng số theo từng cặp khớp, biết mảnh ảnh này mờ hay sắc	Phải huấn luyện; cần chân trị quỹ đạo	Ra ngoài phân phối huấn luyện: trọng số vẫn tự tin, chỉ là sai
Bộ nhớ lượt ngược	—	Trải phẳng: tỉ lệ với tích của số bước, số khung, số kênh và số vị trí. Hàm ẩn: không phụ thuộc số bước	Trải phẳng buộc cửa sổ huấn luyện ngắn, nên không học được hành vi dài hạn
Ổn định số	—	Phải damping, cắt <i>gradient</i> , giới hạn số bước	H gần suy biến: thị sai thấp, quay thuần túy, ảnh mờ — đúng các khung cần học nhất
Chẩn đoán	—	Mất phần lớn	Quỹ đạo lệch mà không có χ^2 , không có số inlier, không có cạnh nào để đổ lỗi
Tính lặp lại	—	Phi tắt định của thư viện lọt vào trong bộ ước lượng	Hai lần chạy cho hai quỹ đạo; hết phân biệt được lỗi thuật toán với nhiễu

Chiều thứ tư đáng nói rõ hơn, vì với một người bảo trì hệ thống nó thường là chiều đắt nhất. Trong backend cổ điển, khi quỹ đạo lệch bạn có một chuỗi câu hỏi rất cụ thể để hỏi. Chi bình phương của cạnh nào lớn bất thường? Bao nhiêu cạnh bị nhân robust ép xuống? Hiệp phương sai biên của khung nào nở ra? Mỗi câu đều có một con số trả lời, và con số đó trở vào một dòng mã. Khi trọng số do một mạng sinh ra, câu hỏi “vì sao nó ra tư thế này” không còn câu trả lời ở mức đồ thị. Bạn vẫn in được phân dư. Bạn không truy được trọng số về nguyên nhân. Và điều tệ nhất là: với bộ giải, một trọng số sai trông y hệt một trọng số đúng.



Hình 1.5. Hai lối đi ngược qua một bộ giải lặp. Trái phằng giữ mọi đại lượng trung gian, nên bộ nhớ tăng tuyến tính theo số bước lặp K ; đổi lại đạo hàm đúng với thuật toán đã chạy, kể cả khi nó chưa hội tụ. Định lý hàm ẩn chỉ cần ma trận H ở bước cuối, nên bộ nhớ không phụ thuộc K ; đổi lại đạo hàm chỉ đúng nếu bước cuối thật sự là một điểm dừng.

1.3 Hạ tầng khả vi: LieTorch, Theseus và BA-Net (Differentiable Infrastructure)

A. LieTorch: gradient sống trong không gian tiếp tuyến

Có một trục trặc rất cụ thể khi ghép autodiff với hình học. PyTorch chỉ biết tensor. Một phép biến đổi $SE(3)$ lưu dưới dạng ma trận 4×4 là 16 số; lưu dưới dạng quaternion cộng tịnh tiến là 7 số. Nhưng nhóm chỉ có 6 bậc tự do. Nếu để autodiff chạy trên biểu diễn nhúng, *gradient* có 16 hoặc 7 thành phần, và phần lớn trong đó trở ra ngoài đa tạp. Người ta chữa bằng cách chuẩn hoá lại quaternion sau mỗi bước, hoặc phạt độ lệch trực giao. Cả hai đều là vá vúi, và cả hai đều làm *gradient* sai một lượng không kiểm soát được.

LieTorch [5] sửa từ gốc. Nó lưu và lan truyền *gradient* trong *không gian tiếp tuyến*, tức đại số Lie. Một phép biến đổi vẫn được lưu bằng 7 số, nhưng đạo hàm của nó là một vectơ 6 chiều, và quy tắc chuỗi được viết lại bằng các Jacobian trái và phải của nhóm. Đó đúng là những Jacobian bạn đã gặp ở Barfoot [6]. Hệ quả thực dụng: không còn chuẩn hoá giữa đồ thị, số chiều *gradient* đúng bằng số bậc tự do, và bước cập nhật là một phép exp chứ không phải một phép cộng.

Bài học thiết kế lấy được ngay cả khi không dùng thư viện: **ghép một mạng vào một khối hình học thì hãy kiểm tra ngay số chiều của gradient**. Nếu nó lớn hơn số bậc tự do thì có một phần *gradient* đang chảy vào các hướng vô nghĩa. Phần đó sẽ hiện ra dưới dạng huấn luyện chậm hoặc không ổn định.

B. Theseus: bắt bạn phải chọn kiểu lượt ngược

Theseus [7] đóng gói cả một bộ giải phi tuyến thành một tầng PyTorch. Bạn khai báo biến và hàm chi phí, nó chạy Gauss-Newton hoặc Levenberg-Marquardt với bộ giải thừa, và nó khả vi. Điều đáng học không nằm ở sự tiện lợi mà ở chỗ nó *bắt người dùng khai báo kiểu lượt ngược*: trái phằng đầy đủ, trái phằng cắt ngắn, hay hàm ẩn. Thư viện không chọn hộ, vì không lựa chọn nào đúng cho mọi bài toán.

Đó là một tín hiệu trung thực đáng quý. Nó nói thẳng rằng đạo hàm qua một bộ giải là một *xấp xỉ có thiên lệch*, và bạn phải chọn nhận thiên lệch nào. Trước Theseus, gradSLAM [8] đã viết lại cả một đường ống SLAM dày đặc dưới dạng khả vi, chủ yếu để chứng minh rằng việc đó làm được.

Khác biệt giữa hai kiểu lượt ngược gọn tới mức viết ra được trong mười lăm dòng giả mã.

```
# Kiểu 1: trái phằng. Autodiff tu lo, nhưng giu moi buoc trong bo nho.
x = x0
for k in range(K):
    H, g = build_normal_equations(x, theta) # do thi phép tính lon dan
    x = x - solve(H + lam*I, g)
L = loss(x); L.backward() # bo nho ~ K

# Kiểu 2: ham an. Chay bo giai KHONG ghi do thi, roi noi lai bang mot lop tu viet.
with torch.no_grad():
    x = newton_solve(x0, theta, K) # bo nho ~ 1
H, B = hessian_and_cross(x, theta) # chi tinh tai nghiem cuoi
v = solve(H, grad_of_loss_wrt_x(x)) # dung lai phan tich cua luot xuai
grad_theta = -(B.t() @ v) # dinh ly ham an
```

Dòng `torch.no_grad()` chính là chỗ đánh đổi. Nó xoá đồ thị phép tính của vòng lặp, nên bộ nhớ tụt về hằng số. Đổi lại, mọi thứ đã xảy ra bên trong vòng lặp trở nên vô hình với **gradient**: bao nhiêu bước đã chạy, damping đã đổi thế nào, có bước nào bị từ chối không. Bảng 1.3 nói rõ ba lựa chọn và chỗ hỏng của từng cái.

Bảng 1.3. Ba kiểu lướt ngược qua một bộ giải lặp. Không kiểu nào đúng cho mọi bài toán, nên thư viện bắt bạn chọn — và lựa chọn đó phải được ghi lại cùng kết quả, giống như số seed.

Kiểu	Bộ nhớ	Đạo hàm là của cái gì	Hỏng khi nào
Trái phằng đầy đủ	Tỉ lệ với số bước K	Đúng thuật toán đã chạy	K lớn hoặc bài toán lớn: hết bộ nhớ, và gradient nhân dồn qua K tầng nên tắt hoặc nổ
Trái phằng cắt ngắn	Tỉ lệ với vài bước cuối	Xấp xỉ, thiên về hành vi cuối vòng lặp	Khởi tạo tồi: phân quan trọng của quỹ đạo tối ưu nằm ở các bước đầu, và bạn vừa cắt bỏ nó
Hàm ẩn	Hằng số	Nghiem của bài toán <i>giả sử đã hội tụ</i>	Bộ giải bị cắt ngắn, hoặc H suy biến vì chưa neo gốc toạ độ

Một quy tắc thực dụng rút ra từ bảng trên. Nếu bộ giải của bạn hội tụ tốt và bạn neo gốc tử tế thì dùng hàm ẩn, vì nó rẻ hơn hẳn. Nếu bộ giải bị ngân sách thời gian ép dừng sớm — trường hợp thường gặp trong hệ thời gian thực — thì trái phằng cắt ngắn trung thực hơn, dù đắt. Và dù chọn cái nào, hãy ghi lựa chọn ấy vào cấu hình thí nghiệm. Nó ảnh hưởng tới kết quả ngang một siêu tham số.

C. BA-Net: học hệ số damping thay vì tự chỉnh

BA-Net [9] là công trình đầu tiên đặt nguyên một lớp **hiệu chỉnh chùm tia** (*bundle adjustment*) vào trong mạng và huấn luyện xuyên qua nó. Nó có ba ý tưởng, và giá trị của ba ý rất khác nhau.

- **Sai lệch trên đặc trưng thay vì trên độ sáng.** Phương pháp trực tiếp cổ điển tối ưu sai khác độ sáng, nên nó thừa hưởng giả định độ sáng không đổi và có lòng chảo hội tụ rất hẹp. BA-Net tối ưu trên các bản đồ đặc trưng học được. Bề mặt lỗi trơn hơn, lòng chảo rộng hơn, và kết quả bất biến với đổi sáng. Đây là ý tưởng bền nhất trong ba ý.
- **Damping học được.** Levenberg-Marquardt cổ điển chỉnh λ bằng luật kinh nghiệm: bước tốt thì chia mười, bước xấu thì nhân mười. BA-Net thay luật đó bằng một mạng nhỏ, đọc phần dư hiện tại rồi xuất ra một vectơ λ cho từng tham số. Vì λ vào bước cập nhật một cách trơn, cả bước LM vẫn khả vi, nên chính sách damping được huấn luyện theo lỗi tư thế cuối.
- **Độ sâu theo cơ sở.** Thay vì coi mỗi điểm ảnh là một ẩn, một mạng sinh ra khoảng một trăm bản đồ độ sâu cơ sở, và ẩn thật sự là các hệ số tổ hợp. Ý này là tổ tiên trực tiếp của CodeSLAM ở mục 6. Số phận của nó cũng báo trước số phận của CodeSLAM: cơ sở độ sâu là một tiên

nghiệm học được về hình dạng cảnh, nên nó tốt đúng trong phân phối huấn luyện và bịa ra bề mặt ở ngoài đó.

Có đáng không? Ý thứ nhất đáng, và quan trọng hơn là nó tách rời được: bạn có thể dùng đặc trưng học được làm mục tiêu căn chỉnh cho một phương pháp trực tiếp mà không cần bất cứ thứ gì khác của BA-Net. Dòng GN-Net đi đúng hướng này, huấn luyện đặc trưng sao cho bề mặt Gauss-Newton quanh nghiệm đúng là một lòng chảo sạch.

Ý thứ hai thì phải nghi ngờ. Một chính sách damping học được vẫn là *chính tay*, chỉ khác ở chỗ hằng số bây giờ được học từ một tập dữ liệu thay vì được gõ vào file cấu hình. Câu hỏi kiểm chứng đúng đắn không phải “nó có tốt hơn trên tập này không”. Câu hỏi đúng là “nó có còn tốt hơn trên một chuỗi có động lực khác hẳn không”. Mình chưa thấy bằng chứng mạnh cho về sau.

CẠM BẤY

Đừng nhầm *bộ giải khả vi* với *bộ giải tốt hơn*. Một lớp Gauss-Newton khả vi hội tụ đúng bằng tốc độ của Gauss-Newton thường, vì ở lượt xuôi nó *chính là* Gauss-Newton thường. Toàn bộ giá trị nằm ở lượt ngược, và lượt ngược chỉ có ý nghĩa nếu bạn thật sự huấn luyện một thứ gì đó bằng nó. Cấm một lớp tối ưu khả vi vào hệ thống rồi không huấn luyện gì cả thì bạn vừa đổi một bộ giải C++ đã tối ưu lấy một bộ giải Python chậm hơn, mà không nhận lại gì.

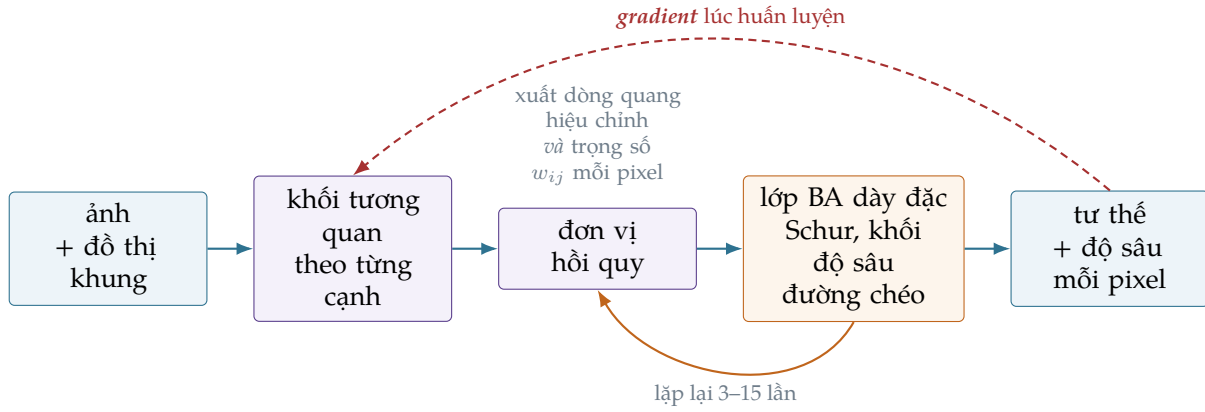
1.4 DROID-SLAM và cuộc thừa hoá sau nó (DROID-SLAM and the Sparsification Line)

A. Cơ chế của DROID-SLAM

DROID-SLAM [10] là hệ đầu-cuối thuyết phục nhất tính tới nay, và cấu tạo của nó đáng vẽ ra. Nó ghép hai thứ đã có: toán tử cập nhật hồi quy của RAFT [11] và một lớp hiệu chỉnh chùm tia dày đặc.

Vòng lặp chạy như sau. Hệ giữ một *đồ thị khung*, với nút là khung then chốt và cạnh là các cặp có chồng lẫn thị giác. Với mỗi cạnh, một khối tương quan được dựng từ hai bản đồ đặc trưng. Một đơn vị hồi quy đọc khối tương quan cùng dòng quang hiện tại rồi xuất ra hai thứ: một mục tiêu dòng quang đã hiệu chỉnh cho từng điểm ảnh, và một trọng số tin cậy cho từng điểm ảnh. Lớp hiệu chỉnh chùm tia dày đặc nhận hai thứ đó làm phép đo, rồi giải bài toán bình phương tối thiểu theo tư thế và theo nghịch đảo độ sâu từng điểm ảnh. Vì khối độ sâu là ma trận đường chéo, phần bù Schur cho ra một hệ camera rút gọn rất nhỏ. Nghiệm mới được đưa ngược lại cho đơn vị hồi quy, và vòng lặp tiếp tục.

Huấn luyện chạy trên TartanAir [12] với chân trị tư thế. *Gradient* đi xuyên qua lớp hiệu chỉnh chùm tia, về tới tận mạng đặc trưng.



Hình 1.6. Vòng lặp của DROID-SLAM. Điểm cốt lõi không phải mạng hồi quy, cũng không phải bộ giải, mà là *vòng phản hồi màu cam* giữa hai cái đó: mạng nhìn thấy nghiệm mà bộ giải vừa trả về rồi sửa phép đo cho lần sau. Mùi tên đứt là đường *gradient* lúc huấn luyện. Nhờ nó, mạng học cách xuất ra dòng quang và trọng số mà bộ giải dùng được, chứ không phải dòng quang trông đẹp mắt.

B. Vì sao một hệ 144 nghìn ẩn vẫn giải được

Với cửa sổ 30 khung then chốt ở $1/8$ độ phân giải, lớp hiệu chỉnh chùm tia này có khoảng 144 nghìn ẩn hình học. Con số đó nghe như một hệ không giải nổi trong thời gian thực. Lý do nó vẫn giải được là một tính chất cấu trúc bạn đã quen, chỉ khác quy mô.

Trong hiệu chỉnh chùm tia cổ điển, mỗi điểm bản đồ đóng góp một khối 3×3 trên đường chéo của khối cấu trúc, nên khử chúng bằng phân bù Schur là rẻ. Ở đây còn dễ hơn. Mỗi ẩn là *một* nghịch đảo độ sâu của *một* điểm ảnh, và nó không nối trực tiếp với điểm ảnh nào khác. Khối cấu trúc vì thế là một ma trận đường chéo thật sự, không phải khối đường chéo. Nghịch đảo nó là 144 nghìn phép chia.

Sau khi khử, hệ còn lại chỉ là hệ camera rút gọn cỡ $6N \times 6N$. Với $N = 30$ khung then chốt thì đó là ma trận 180×180 . Phân tích Cholesky một ma trận cỡ đó là chuyện vặt so với phần còn lại của vòng lặp. Nói cách khác, **phần đắt của DROID-SLAM không nằm ở bộ giải**. Nó nằm ở việc dựng khối tương quan rồi chạy mạng hồi quy để sinh ra 144 nghìn phép đo cùng 144 nghìn trọng số, lặp lại tới mười lăm lần cho mỗi khung.

Kết luận thực dụng rút ra khá rõ: nếu định thử một ý tưởng theo hướng này thì đừng tối ưu bộ giải. Ba chỗ cắt được là độ phân giải, số bước lặp, và số cạnh trong đồ thị khung. Cả ba đều là tham số cấu hình chứ không phải kiến trúc, nên thử rất nhanh.

C. Vì sao nó chính xác

Bốn cơ chế cộng lại, và chỉ một trong bốn liên quan tới chữ “khả vi”.

- **Dày đặc là dư thừa.** Mỗi điểm ảnh bỏ một lá phiếu, nên vài nghìn phiếu sai không lay chuyển được nghiệm. Một hệ thừa chỉ có vài trăm cặp khớp thì có.
- **Trọng số theo từng điểm ảnh.** Mạng hạ trọng số vùng trời, vật thể chuyển động, bề mặt phản chiếu. Kiểm tra tỉ lệ khoảng cách mô tả không làm được việc đó, vì nó không nhìn ngữ cảnh.
- **Sửa lặp lại.** Đơn vị hồi quy thấy nghiệm hiện tại rồi sửa phép đo, nên hệ tự gỡ được khởi tạo tồi. Một đường ống một chiều không có cơ chế này.
- **Hình học vẫn được ép.** Đầu ra đi qua một bộ giải thật, nên nó nhất quán đa khung. Nó không trôi theo kiểu một mạng hồi quy tư thế.

Cơ chế thứ ba mới là chỗ tính khả vi trả tiền. Không có *gradient* xuyên qua bộ giải thì mạng sẽ được huấn luyện để dự đoán dòng quang khớp với chân trị dòng quang. Có *gradient*, nó được huấn luyện để dự đoán dòng quang làm cho *tư thế* đúng. Hai mục tiêu đó khác nhau, và khác biệt nằm đúng ở các điểm ảnh có hình học xấu.

D. Vì sao nó đắt

Đếm ẩn số trước. Ảnh 640×480 ở $1/8$ độ phân giải cho 4 800 ẩn nghịch đảo độ sâu cho mỗi khung then chốt. Với 30 khung trong cửa sổ là 144 000 ẩn hình học, so với 6 ẩn tư thế mỗi khung. Bài toán vẫn giải được vì khối độ sâu là đường chéo, nhưng mọi tensor đều mang kích thước đó.

Đếm bộ nhớ sau, và đếm bằng công thức chứ không quy ra một con số. Một khối tương quan toàn phần giữa hai khung giữ một mục cho *mỗi cặp* vị trí, tức bình phương số vị trí ở độ phân giải đang chạy — và đó mới là một cạnh, một mức. Đồ thị khung có nhiều cạnh cho mỗi nút, nên tổng bộ nhớ tỉ lệ với số cạnh nhân bình phương số vị trí. Chữ “bình phương” đó giải thích vì sao bộ nhớ tăng rất nhanh theo độ phân giải, vì sao nó tăng theo độ dài chuỗi, và vì sao các công trình sau phải thêm cơ chế quản lý bộ nhớ.

Còn phần cứng. Các tác giả công bố hệ trên GPU để bàn cao cấp, không phải trên máy nhúng. Một Jetson Orin có ngân sách tính toán và băng thông bộ nhớ thấp hơn hẳn một GPU để bàn cùng thế hệ, và khoảng cách ấy thuộc loại bậc độ lớn chứ không phải vài chục phần trăm. Đây là hướng, không phải số đo: mình không có phép đo nào cho phát biểu này, và muốn một con số thì phải tự chạy. Thêm nữa, DROID-SLAM không dùng IMU, tức là bỏ đi tài sản mạnh nhất của một hệ stereo-inertial.

E. Một bảng tính ngân sách Orin, làm trong mười phút

Trước khi tải bất cứ mã nguồn nào về, có một việc rẻ hơn nhiều: đếm phép tính trên giấy. Nó không cho bạn con số đúng, nhưng nó cho bạn biết bạn đang hụt *hai lần* hay *hai mươi lần*. Hai kết luận đó dẫn tới hai quyết định hoàn toàn khác nhau.

Giả sử mục tiêu là vài chục khung mỗi giây, và frontend cổ điển của bạn đã ăn một phần đáng kể ngân sách mỗi khung. Bảng 1.4 liệt kê các khoản mục, cách tính từng khoản, và mức nặng nhẹ tương đối giữa chúng.

Bảng 1.4. Các khoản mục ngân sách của một hệ đầu-cuối kiểu DROID trên phần cứng nhúng. Cột giữa là công thức đếm; cột cuối là *xếp hạng định tính* giữa các khoản, kèm lý do của thứ hạng đó. Bảng này cố tình không có con số: nó dùng để biết nên cắt ở đâu, còn độ lớn thật thì chỉ phép đo trên chính thiết bị của bạn mới nói được.

Khoản mục	Cách tính	Thứ hạng, và vì sao
Mạng trích đặc trưng	Đếm phép nhân cộng của backbone ở độ phân giải đầu vào thật, không phải ở 224×224	Nặng. Chi phí tỉ lệ thẳng với số điểm ảnh, nên đo ở độ phân giải trong bài rồi suy ra độ phân giải thật là cách hệt nhiều lần
Khối tương quan	$(H/8)(W/8)$ nhân $(H/8)(W/8)$ cho mỗi cạnh, nhân số byte	Nặng nhất về bộ nhớ. Đây là khoản duy nhất tăng theo <i>bình phương</i> số vị trí, lại còn nhân với số cạnh của đồ thị khung
Vòng lặp hồi quy	Nhân chi phí một bước với số bước, thường 3 tới 15	Khoản đội chi phí lớn nhất, vì nó nhân thẳng mọi khoản phía trên lên vài lần tới hơn chục lần
Giải hệ BA	Hệ camera rút gọn $6N \times 6N$, với N là số khung trong cửa sổ	Nhẹ nhất, và hay bị đổ oan. Nó nhỏ hơn phần sinh phép đo nhiều bậc, nên tối ưu nó gần như không đổi được gì
Chuyển dữ liệu	Tổng byte đi qua bộ nhớ chia cho băng thông thật của thiết bị	Thường mới là nút cổ chai thật, và chỉ lộ ra khi bạn đếm byte thay vì đếm phép tính

Cách dùng bảng quan trọng ngang bản thân bảng. Nhân các khoản theo đúng công thức ở cột giữa, chia cho thông lượng *đo được* của thiết bị — không phải con số đỉnh trên tờ thông số, vốn luôn cao hơn nhiều — rồi so với ngân sách mỗi khung. Thứ bạn cần lấy ra từ phép tính đó không phải một con số mili-giây, mà đúng một bit thông tin: bạn đang hực *một chút* hay hực *một bậc độ lớn*. Hực một chút thì đi đo thật, vì phép tính trên giấy không đủ chính xác để kết luận. Hực một bậc thì đừng đo vội, vì kết luận đã rõ rồi: phải cắt hẳn một thừa số trước đã. Và đúng ba chỗ cắt được là: hạ độ phân giải, giảm số bước lặp, hoặc giảm mật độ. Mục kế tiếp là câu chuyện của lựa chọn thứ ba.

↔ **cùng nguyên lý với** B/11-gioi-han-tinh-toan — đây chính là bài toán roofline của chương 11 áp cho một hệ SLAM: đếm phép tính và đếm byte trước, đo sau

F. Thừa hoá: DPVO, DPV-SLAM và MAC-VO

Nếu chỗ đất là mật độ, thì câu hỏi tự nhiên là mật độ có cần thiết không.

I Mạch — từ dày đặc về thưa

1 Vấn đề

DROID-SLAM chính xác nhưng cần phần cứng GPU để bàn cao cấp và một lượng bộ nhớ vượt xa máy nhúng, nên không đưa lên robot được.

Giải pháp

DPVO [13] bỏ trường dòng quang dày đặc. Thay vào đó nó theo vết một tập nhỏ các *mảnh ảnh* vuông, mỗi khung khoảng một trăm mảnh, mỗi mảnh mang đúng một ẩn nghịch đảo độ sâu. Vẫn cùng toán tử hồi quy và cùng lớp hiệu chỉnh chùm tia, chỉ khác là chạy trên đồ thị mảnh.

Giả định

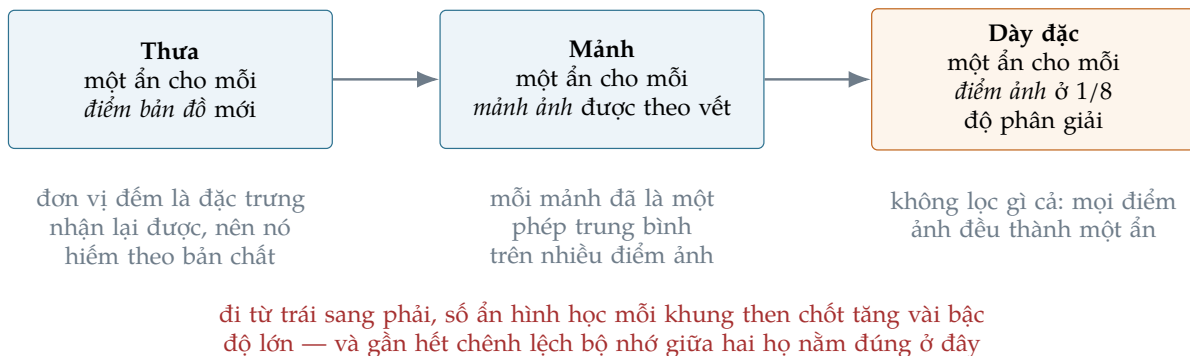
Ích lợi thống kê của việc dày đặc bão hoà rất sớm, vì bản thân một mảnh đã là một phép trung bình trên nhiều điểm ảnh.

Hệ quả

Số ẩn hình học mỗi khung giảm vài bậc độ lớn, và theo báo cáo của chính các tác giả thì hệ nhẹ đi đủ nhiều để chạy trên một GPU để bàn duy nhất thay vì nhiều card. Minh không kiểm chứng được các con số tốc độ và bộ nhớ họ đưa ra, nên ở đây chỉ giữ lại hướng của thay đổi.

Giá phải trả	Mất khả năng tự nhiên cho ra bản đồ dày đặc, và mất một phần sức chống chịu khi cảnh có rất ít vân.
2 Vấn đề	DPVO là odometry: không có bản đồ toàn cục, nên nó trôi và không bao giờ đóng vòng được.
Giải pháp	DPV-SLAM [14] thêm hai cơ chế đóng vòng. Một cơ chế dựa trên chính đồ thị mảnh cho các vòng gần, và một cơ chế truy hồi ảnh kiểu cổ điển cho các vòng xa. Sau đó chạy tối ưu đồ thị tư thế toàn cục.
Hệ quả	Thành một hệ SLAM đầy đủ, vẫn gần thời gian thực trên một GPU.
Giả định	Bài học còn quan trọng hơn kết quả: <i>chưa ai học được cách thay thế đóng vòng và tối ưu đồ thị tư thế</i> . Ngay cả một hệ đầu-cuối cũng phải mượn lại đúng hai khối cổ điển đó.
3 Vấn đề	Trọng số vô hướng cho mỗi cặp khớp vẫn quá thô. Bất định của một cặp khớp stereo có hình dạng rất dẹt: dài theo tia nhìn và ngắn theo phương ngang.
Giải pháp	MAC-VO [15] dự đoán cả một hiệp phương sai cho từng cặp khớp. Quan trọng là hiệp phương sai ấy <i>có đơn vị mét</i> , tức nó tính tới việc sai số độ sâu nở ra khi disparity nhỏ. Con số đó được dùng hai lần: để chọn điểm nào đáng giữ, và để làm ma trận thông tin trong bài toán tư thế.
Hệ quả	Đây là cầu nối trực tiếp sang F/34-imu-bat-dinh-va-trien-khai. Thứ được học không còn là đặc trưng hay dòng quang, mà chính là <i>ma trận thông tin</i> — đúng đại lượng bạn đang đặt bằng tay.
Giá phải trả	Vẫn cần huấn luyện có chân trị, và bất định học được kế thừa mọi giới hạn phân phối ở mục 5. Chi tiết số của công trình này mình chưa tự kiểm chứng, nên chỉ nêu ý tưởng.

Hình 1.7 đặt ba mức mật độ cạnh nhau để thấy khoảng cách thật sự lớn tới đâu.



Hình 1.7. Ba mức mật độ, đọc theo đơn vị đếm chứ không theo con số. Điều đáng nhớ không phải mỗi họ giữ bao nhiêu ẩn, mà là mỗi họ lấy gì làm đơn vị: một hệ thưa đếm theo điểm bản đồ nhận lại được, DPVO đếm theo mảnh ảnh, DROID đếm theo điểm ảnh. Vì mỗi bậc trong ba bậc đó cách nhau vài bậc độ lớn về số lượng, thứ tự này quyết định gần như toàn bộ chênh lệch bộ nhớ giữa hai họ — nhiều hơn hẳn mọi khác biệt về cài đặt.

G. Đánh giá theo bốn câu hỏi

Nó giải quyết việc gì. Đóng góp thật của dòng này là trọng số theo từng cặp khớp và vòng sửa lặp lại. Hai thứ đó chạm đúng vào chuyển động nhanh và ảnh mờ. Đó là các trường hợp mà kiểm tra tỉ lệ khoảng cách mô tả cho ra hoặc quá ít cặp, hoặc quá nhiều cặp sai.

Có đáng làm không. Với một hệ stereo-inertial đã tinh chỉnh chạy trên Orin thì chưa. Ba lý do, theo thứ tự nặng dần. Ngân sách tính toán chênh một bậc độ lớn. Toàn bộ dòng này bỏ IMU, nên

bạn sẽ đánh đổi thứ đang cứu bạn ở những khung mờ để lấy thứ hứa cứu bạn ở những khung mờ. Và tính lặp lại sứt — một hệ bạn không chẩn đoán được là một hệ bạn không bảo trì được.

Học gì từ thiết kế. Ba bài học tách rời được, xếp theo mức dễ áp dụng. Dùng trọng số liên tục thay cho quyết định inlier nhị phân. Đưa nghiệm hiện tại quay lại cho tầng sinh phép đo, thay vì chạy một chiều. Và giữ đồ thị khung như một cấu trúc dữ liệu tường minh, thay vì để nó ẩn trong danh sách kề của bản đồ.

→ **tiền đề** F/30-do-sau-dong-quang-va-mo-hinh-nen — chất lượng của cả họ này bị chặn trên bởi chất lượng dòng quang, tức bởi đúng dòng RAFT mà chương 30 bàn

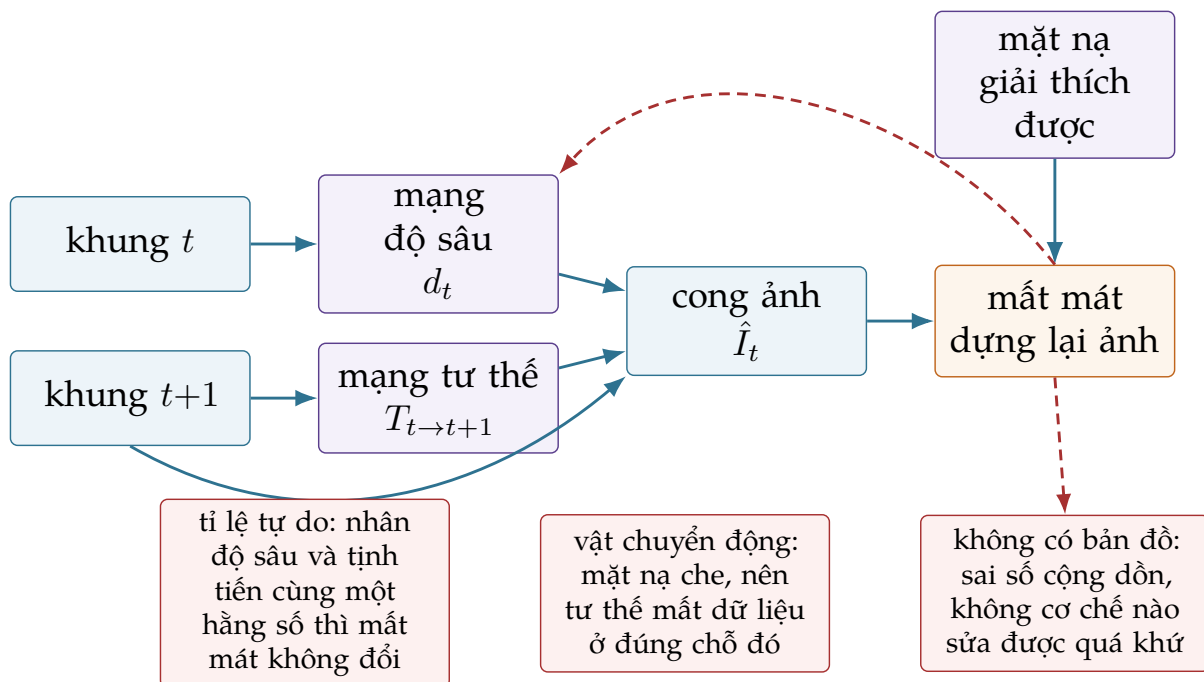
1.5 VO tự giám sát và giới hạn thật của nó (Self-Supervised VO)

A. Bài toán và cơ chế của SfM-Learner

Chân trị tư thế đắt. Cần một hệ đo động ngoài, hoặc một buổi chạy COLMAP dài. Câu hỏi của SfM-Learner [16] là: có thể huấn luyện chỉ bằng video thô không?

Cơ chế gọn và đẹp. Hai mạng cùng học. Mạng độ sâu nhận một khung và xuất bản đồ độ sâu. Mạng tư thế nhận một cặp khung và xuất phép biến đổi giữa chúng. Có độ sâu, có tư thế, có nội tham số camera thì *dựng lại* được khung đích từ khung nguồn bằng phép cong ảnh. **Hàm mất mát** (*loss function*) là chênh lệch giữa ảnh dựng lại và ảnh thật. Không cần nhãn nào.

Chi tiết cấu cả ý tưởng là *mặt nạ giải thích được*. Đó là một đầu ra thứ ba, một con số mỗi điểm ảnh, cho phép mạng nói “điểm này tôi không giải thích được, đừng phạt tôi”. Nó xử lý vật thể chuyển động, vùng bị che, vùng ra ngoài khung. Nhưng nó có một nghiệm suy biến hiển nhiên: đặt mặt nạ bằng 0 ở mọi nơi thì mất mát bằng 0. Nên phải cộng một số hạng chính quy kéo mặt nạ về 1. Điểm này đáng ghi nhớ, vì nó là mẫu chung của mọi thiết kế tự giám sát: **cho mô hình quyền bỏ qua dữ liệu thì phải bắt nó trả tiền cho quyền đó**.



Hình 1.8. Vòng huấn luyện tự giám sát kiểu SfM-Learner, và ba chỗ hỏng gắn liền với nó. Tín hiệu dạy duy nhất là “dựng lại khung t từ khung $t+1$ thì phải khớp”. Ba hộp đỏ không phải lỗi cài đặt mà là hệ quả trực tiếp của chính hàm mất mát đó: mặt nạ mua sự bền vững bằng cách vứt dữ liệu, tỉ lệ không quan sát được từ ảnh đơn, và không có trạng thái nào để một lần quan sát lại có thể sửa quá khứ.

B. Bốn giới hạn, nói thẳng

Đây là mục một kỹ sư SLAM cần đọc kỹ nhất, vì các bảng kết quả trong dòng này rất dễ đọc nhầm.

- **Mơ hồ tỉ lệ.** Với ảnh đơn, nhân độ sâu với s và nhân tịnh tiến với s cho ra đúng cùng một ảnh dựng lại. Mắt mắt không phân biệt được hai trường hợp. Nên đầu ra chỉ đúng tới một hằng số chưa biết, và hằng số đó còn trôi giữa các cảnh. Nhiều bảng KITTI trong dòng này căn tỉ lệ với chân trị *theo từng chuỗi* trước khi tính lỗi. Một robot không làm được việc đó, nên con số ấy không phải con số odometry dùng được. UnDeepVO [17] chữa bằng cách huấn luyện trên cặp stereo — đường cơ sở đã biết bơm tỉ lệ mét vào — rồi chạy đơn ảnh lúc suy luận.
- **Không có bản đồ, nên không có đóng vòng.** Mạng tư thế là phép ước lượng chết theo từng cặp khung. Sai số cộng dồn và không gì kéo nó về được. Toàn bộ độ chính xác của hệ thống của bạn đến từ việc quan sát lại cùng một điểm bản đồ qua nhiều khung then chốt. Ở đây cơ chế đó không tồn tại.
- **Không có bảo đảm, và hỏng êm.** Không hiệp phương sai, không phần dư để kiểm, không cổng χ^2 . Khi sai, nó sai một cách mượt mà và tự tin. So sánh với hệ của bạn: khi sắp mất bám, số inlier tụt trước, và bạn thấy được.
- **Phụ thuộc phân phối huấn luyện.** DeepVO [18] hồi quy tư thế trực tiếp và rất tốt trên KITTI. Một phần lý do là nó học luôn *tiên nghiệm chuyển động của một chiếc ô tô*: đi thẳng, tốc độ trong một dải hẹp, góc lệch nhỏ. Đó là một cơ chế thật chứ không phải lời che. Đưa mô hình sang camera cầm tay hoặc drone thì tiên nghiệm ấy sai, và nó sai theo hướng có hệ thống.

C. Mơ hồ tỉ lệ, nhìn bằng một phép tính

Giới hạn thứ nhất đáng được làm cho cụ thể, vì nó là chỗ nhiều người đọc bảng kết quả bị lừa.

Phép cong ảnh dùng đúng một biểu thức. Điểm ảnh u ở khung nguồn được đưa về khung đích qua độ sâu $d(u)$ và phép biến đổi (R, t) . Bây giờ thay $d \rightarrow s d$ và $t \rightarrow s t$ với cùng một hằng số $s > 0$. Toạ độ chiếu ra không đổi, vì s rút gọn giữa tử và mẫu. Ảnh dựng lại giống hệt, nên mắt mắt giống hệt. Kết luận: hàm mắt mắt *không chứa thông tin* nào về s .

Sự kiện nhỏ này dẫn tới một hệ quả lớn khi đọc bảng. Giả sử — đây là một phép tính giả định, không trích từ bài nào — một hệ báo lỗi tịnh tiến vài phần trăm chiều dài quỹ đạo, sau khi đã căn tỉ lệ theo từng chuỗi. Nếu bạn không được căn tỉ lệ, và tỉ lệ ước lượng lệch mười phần trăm, thì riêng sai lệch tỉ lệ đã sinh ra lỗi bằng mười phần trăm chiều dài quỹ đạo — lớn hơn hẳn con số được báo cáo, và lớn dần theo quãng đường đi. Con số trong bảng khi đó nói về *hình dạng* quỹ đạo, không nói về *vị trí*. Với một robot cần biết mình đang ở đâu, hai đại lượng đó không thay thế nhau được.

Cách kiểm nhanh khi đọc một bài báo trong dòng này chỉ có hai câu hỏi. Tỉ lệ được lấy từ đâu: từ stereo, từ IMU, hay từ chân trị? Và nếu từ chân trị thì căn một lần cho cả chuỗi hay căn lại theo từng đoạn? Căn theo từng đoạn là dấu hiệu tỉ lệ đang trôi, và độ trôi đó chính là thứ bị giấu đi.

D. TartanVO và cách chữa đúng chỗ

TartanVO [19] là câu trả lời có kỷ luật cho gạch đầu dòng cuối. Hai thủ thuật của nó đáng học kể cả khi bạn không dùng mô hình.

Thứ nhất, **bỏ tỉ lệ ra khỏi hàm mất mát**. Ép mạng học tỉ lệ tuyệt đối từ ảnh đơn thì nó buộc phải học tỉ lệ đặc thù của tập dữ liệu. Chuẩn hoá vectơ tịnh tiến trong hàm mất mát thì mạng chỉ còn phải học hướng, và hướng thì tổng quát hoá được.

Thứ hai, **đưa mô hình camera vào đầu vào**. Nối thêm vài kênh mã hoá nội tham số đã chuẩn hoá vào tensor đầu vào. Nhờ đó một mô hình duy nhất phục vụ nhiều camera có tiêu cự và độ méo khác nhau, thay vì phải huấn luyện lại cho từng camera.

Kết quả là một mô hình chạy qua nhiều tập dữ liệu mà không tinh chỉnh lại. Độ chính xác khiêm tốn, nhưng tính tổng quát là thật. Bài học chung: muốn tổng quát hoá thì lấy đại lượng đặc thù tập dữ liệu ra khỏi hàm mất mát, và đưa thông tin thiết bị vào đầu vào.

E. Nhánh giữ lại bộ giải cổ điển

Bốn công trình còn lại trong mục này đáng nhắc vì chúng chọn một kiến trúc khác hẳn. Với người đọc này, đó mới là kiến trúc đáng dùng.

- **DeepTAM** [20] tách tracking và mapping đúng như một hệ cổ điển rồi học từng phần, thay vì trộn cả hai vào một mạng.
- **DeepV2D** [21] luân phiên một khối độ sâu và một khối chuyển động, trong đó khối chuyển động là một bước Gauss-Newton khả vi. Nó là cầu nối giữa mục này và mục 4.
- **MonoRec** [22] học độ sâu đa khung có xử lý vật chuyển động, nhưng *lấy tư thế từ một hệ SLAM cổ điển*. Nó không cố thay bộ ước lượng tư thế.
- **TANDEM** [23] đi xa nhất theo hướng đó. Tracking bằng phương pháp trực tiếp cổ điển, còn phần học được chỉ lo dựng bản đồ dày đặc, rồi hợp nhất vào một trường TSDF.

Bốn công trình trên vẽ ra một mẫu thiết kế rõ ràng: **giữ bộ ước lượng tư thế cổ điển, học phần dày đặc**. Với một hệ stereo-inertial đã chạy tốt, đây là mẫu duy nhất trong cả mục 5 mà mình cho là đáng thử, vì nó không đặt cược thứ đang hoạt động.

I 1.6 Biểu diễn ẩn: nén hình học vào một mã ngắn (Latent Scene Representations)

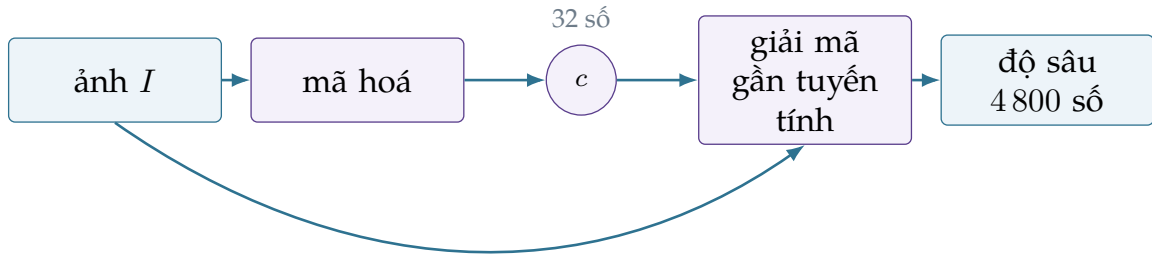
A. Vấn đề mà mã ngắn định giải

Bản đồ độ sâu dày đặc của một khung có cỡ 10^5 ẩn. Đưa từng ấy ẩn vào một bài toán bình phương tối thiểu thì hoặc phải chính quy hoá bằng luật kinh nghiệm, hoặc phải có GPU. Nhưng hình học của một căn phòng thật không có 10^5 bậc tự do. Tường phẳng, sàn phẳng, vật thể có hình dạng quen thuộc. Hình học ấy nằm gần một đa tạp số chiều thấp.

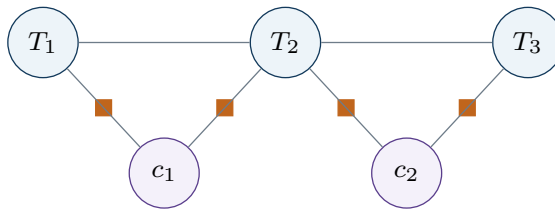
CodeSLAM [24] rút ra kết luận trực tiếp. Học một bộ mã hoá đưa ảnh về một *mã ngắn* c cỡ vài chục chiều, cùng một bộ giải mã $D(c, I)$ trả về bản đồ độ sâu. Rồi đặt c vào đồ thị nhân tử *ngay cạnh các nút tư thế*, và tối ưu chung.

Chi tiết kỹ thuật làm ý tưởng chạy được ít khi được nhắc, nhưng nó là mấu chốt. Bộ giải mã được thiết kế để *gần tuyến tính theo c* khi đã cố định ảnh. Nhờ đó $\partial D / \partial c$ tính một lần cho mỗi khung then chốt rồi dùng lại cho mọi bước Gauss-Newton. Không có tính chất đó thì mỗi lần tuyến tính hoá tốn một lượt ngược qua mạng, và bài toán hết khả thi thời gian thực.

Bộ mã hoá và bộ giải mã



Đồ thị nhân tử



mỗi nút mã thay cho 4800 ẩn độ sâu, nhưng nó nối tới mọi tư thế cùng quan sát khung đó

Hình 1.9. Ý tưởng của CodeSLAM. Bộ giải mã được thiết kế gần tuyến tính theo mã c , nên $\partial D/\partial c$ là một ma trận cố định, tính một lần cho mỗi khung then chốt. Nhờ đó mã trở thành một biến bình thường trong đồ thị nhân tử, đứng ngang hàng với tư thế, và hình học dày đặc được tối ưu chung với quỹ đạo bằng đúng bộ giải bạn đã có.

B. Đếm ẩn và đếm chi phí tuyến tính hoá

Ba con số cho thấy vì sao mã ngăn hấp dẫn tới vậy, và cũng cho thấy chỗ nó âm thầm đắt.

Số ẩn. Một bản đồ độ sâu ở 1/8 của ảnh 640×480 có 4800 ẩn. Một mã 32 chiều thay thế cả từng ấy. Tỉ số nén là 150 lần. Khối cấu trúc trong hệ phương trình chuẩn co từ 4800×4800 đường chéo xuống 32×32 đặc, và biên hoá một khối 32×32 là chuyện vặt.

Chi phí Jacobian. Đây mới là chỗ ẩn chi phí. Ma trận $\partial D/\partial c$ có kích thước 4800×32 , tức khoảng 154 nghìn số. Nếu bộ giải mã không gần tuyến tính theo c thì ma trận đó phải tính lại ở mỗi lần tuyến tính hoá, và mỗi lần tính là 32 lượt ngược qua bộ giải mã. Với năm bước Gauss-Newton là 160 lượt ngược cho mỗi khung then chốt. Không hệ thời gian thực nào chịu nổi.

Vì sao gần tuyến tính lại cứu được. Nếu $D(c, I) \approx D_0(I) + J(I)c$ thì J chỉ phụ thuộc ảnh, nên tính một lần khi khung then chốt được tạo rồi dùng lại mãi. Chi phí mỗi bước Gauss-Newton trở về đúng một phép nhân ma trận, y như một cạnh chiếu lại thông thường. Toàn bộ tính khả thi của CodeSLAM nằm ở dòng vừa rồi. Và giá của nó là điều kiện gần tuyến tính chỉ đúng trong một lân cận nhỏ, tức lý do thứ hai ở mục C bên dưới.

C. Dòng công trình

SceneCode [25] mở rộng mã sang cả nhãn ngữ nghĩa, để hình học và ngữ nghĩa cùng được tinh chỉnh. DeepFactors [26] đưa mã vào một đồ thị nhân tử thật với ba loại nhân tử — quang trắc, chiếu lại, và hình học thưa — rồi chạy được thời gian thực. NodeSLAM [27] đổi phạm vi: mỗi vật thể có một mã hình dạng riêng, và tối ưu chạy trên tư thế cộng hình dạng của từng vật. CodeMapping [28] thực dụng nhất: giữ nguyên ORB-SLAM3 làm xương sống thưa, rồi chạy một tầng bản đồ dày đặc bằng mã ở bên cạnh, có điều kiện trên chính các điểm thưa đó.

D. Vì sao hướng này chưa thẳng

Câu hỏi này giá trị hơn danh sách trên. Ý tưởng đẹp, cài đặt chạy được, người làm giỏi. Vậy tại sao tám năm sau nó vẫn không nằm trong hệ sản xuất nào? Bốn lý do, xếp theo mức nền tảng.

Thứ nhất, sai số của tiên nghiệm có tương quan chứ không thưa. Toàn bộ phòng thủ của ước lượng robust dựa trên một giả định: điểm ngoại lai thì ít và độc lập với nhau. Một bộ giải mã huấn luyện trên ảnh RGB-D trong nhà không sinh ra sai số như vậy. Nó sinh ra những bề mặt trơn, hợp lý, sai một cách có hệ thống, và *nhất quán giữa các khung* vì cùng một bộ giải mã tạo ra chúng. Với một cổng χ^2 , thứ đó trông y hệt một bức tường thật. Đây là lý do sâu nhất, và nó áp cho mọi tiên nghiệm học được chứ không riêng mã hình học.

Để thấy điều đó, hãy so hai kiểu lỗi trên cùng một cổng χ^2 , bằng một ví dụ *giả định* — không phải số đo trên hệ nào. Một cặp khớp sai lệch cỡ vài chục điểm ảnh cho χ^2 vọt lên hàng trăm, nên cổng 5,991 loại nó ngay ở khung đầu tiên. Bây giờ giả sử bộ giải mã dựng một bề mặt lệch ΔZ vài chục centimet ở khoảng cách vài mét. Đưa độ lệch ấy qua công thức chiếu $\Delta u = f b \Delta Z / Z^2$, với f và b của một cấu hình stereo cỡ EuRoC, phần dư thu được nhỏ hơn một điểm ảnh — vì Z^2 ở mẫu số nuốt gần hết độ lệch. Nó lọt qua mọi cổng. Tệ hơn, nó lọt qua ở *mọi* khung cùng một kiểu, nên các khung củng cố lẫn nhau và bộ ước lượng càng lúc càng tự tin vào một bức tường không có thật.

Thứ hai, bán kính hiệu lực của phép tuyến tính hoá rất nhỏ. Bộ giải mã chỉ gần tuyến tính *quanh mã mà bộ mã hoá vừa xuất ra*. Một hiệu chỉnh lớn sẽ đi ra ngoài vùng đó, và $\partial D / \partial c$ không còn đúng. Nói cách khác, tối ưu chỉ sửa được những sai lệch nhỏ — đúng trường hợp mà dự đoán ban đầu vốn đã tốt.

Thứ ba, độ phủ. Ba mươi hai chiều đủ cho phân phối huấn luyện và không đủ cho thứ khác. Ngoài trời, ban đêm, một sàn nhà máy đầy kim loại phản chiếu — bộ giải mã không có cơ sở nào để dựng bề mặt ở đó, nhưng nó vẫn dựng ra một bề mặt.

Thứ tư, phần thưởng đã dờn chỗ. Năm 2018, đây gần như là con đường duy nhất để có bản đồ dày đặc tối ưu được. Tới 2023, 3D Gaussian Splatting [29] cho một bản đồ dày đặc, tinh chỉnh tăng dần được, chất lượng ảnh cao hơn hẳn, mà không cần bất cứ tiên nghiệm học được nào về hình dạng. Đó là nội dung của F/32-ban-do-neural. Thêm nữa, xét riêng độ chính xác quỹ đạo, dòng mã ngắn chưa bao giờ vượt được SLAM đặc trưng thưa. Bản đồ mới là sản phẩm, còn tư thế chỉ là sản phẩm phụ.

E. Cái còn lại

Trước khi rời mục này, đáng chốt lại một bài kiểm tra hai câu, dùng được cho *mọi* tiên nghiệm học được mà bạn định cắm vào backend — mã hình học, độ sâu đơn ảnh, hay pháp tuyến bề mặt.

Câu thứ nhất: sai số của nó có độc lập giữa các khung không? Nếu cùng một mạng sinh ra dự đoán cho mọi khung thì câu trả lời gần như luôn là không. Khi đó nhân robust và cổng χ^2 mất tác dụng, và bạn phải hạ trọng số của cả nhóm phép đo ấy bằng tay, chứ không thể trông vào bộ ước lượng tự loại.

Câu thứ hai: sai số của nó có biểu hiện thành phần dư lớn không? Đây là câu hỏi định lượng, và mục trước đã cho cách tính: đổi độ lệch hình học ra phần dư điểm ảnh bằng công thức chiếu. Nếu một sai lệch nguy hiểm chỉ tạo ra phần dư dưới một điểm ảnh thì không cổng nào bắt được nó, và tiên nghiệm ấy phải được cấp một hiệp phương sai lớn ngay từ đầu.

Hai câu đó không cần thí nghiệm nào. Chúng trả lời được trên giấy trong mười phút, và chúng loại đi phần lớn ý tưởng nghe hay.

Dù vậy có một mảnh đáng giữ, và nó không phải cái mã. Đó là ý tưởng đặt một biến ẩn số chiều thấp vào đồ thị nhân tử rời ước lượng luôn hiệp phương sai của nó. Dạng khoẻ mạnh nhất của ý tưởng ấy là NodeSLAM. Một mã hình dạng cho *một loại vật thể cụ thể* là tiên nghiệm được xác

định rõ hơn nhiều so với một mã cho “một căn phòng”. Tiên nghiệm càng hẹp thì càng ít cơ hội bịa ra bề mặt.

CẦN NHỚ

Trong một bộ ước lượng bình phương tối thiểu, thứ nguy hiểm không phải là tiên nghiệm học được *sai*, mà là nó sai *một cách trơn tru và nhất quán giữa các khung*. Nhân robust, cổng χ^2 và RANSAC đều được thiết kế cho ngoại lai thừa và độc lập. Không cái nào bắt được một bề mặt bịa ra mà mọi khung đều đồng ý. Trước khi cắm bất cứ đầu ra học được nào vào backend, hãy hỏi đúng một câu: *sai số của nó có độc lập giữa các khung không*. [T1]

1.7 Giới hạn và trường hợp hỏng

A. Giới hạn của chính ý tưởng khả vi

Chế độ hỏng nền tảng nhất đã nêu ở mục 2 và đáng nhắc lại thành một câu: ***gradient chỉ chảy qua những thứ vốn đã liên tục***. Trong một hệ SLAM đầy đủ, các quyết định đắt giá nhất lại rời rạc. Chèn hay không chèn khung then chốt. Chấp nhận hay từ chối một ứng viên đóng vòng. Gộp hay không gộp hai bản đồ. Không cái nào có đạo hàm. Nên cụm từ “đầu-cuối” luôn hẹp hơn nghĩa đen của nó. Phần được học chính là phần vốn đã trơn, còn xương sống rời rạc vẫn do luật viết tay quyết định — và đó thường là phần gây ra các lần hỏng nặng nhất.

Chế độ hỏng thứ hai nằm ở ổn định số. H^{-1} nở ra khi H gần suy biến, tức khi thị sai thấp, khi camera quay gần như thuần túy, khi ảnh mờ. Đó chính xác là những khung bạn muốn mạng học từ chúng. Nên ***gradient*** lớn nhất và nhiều nhất lại đến từ các mẫu quý nhất. Mọi cách chữa — tăng damping, cắt ***gradient***, giảm số bước lặp — đều làm thiên lệch đạo hàm theo hướng làm nhẹ đúng các mẫu đó.

Chế độ hỏng thứ ba là hàm mất mát. ***Gradient*** chỉ tốt bằng đại lượng bạn đặt ở cuối. Huấn luyện trên các đoạn video ngắn với lỗi tư thế theo cặp khung thì chỉ tối ưu tính nhất quán ngắn hạn. Nó không tối ưu độ trôi sau ba trăm mét, cũng không tối ưu xác suất đóng vòng đúng. Hai thứ đó lại chiếm phần lớn sai số thật của bạn. Đây là biến thể của bài học đo lường ở E/24-thi-nghiem-va-ablation: tối ưu cái bạn đo được thay vì cái bạn quan tâm là một cái bẫy, không phải một chiến lược.

B. Giới hạn của các hệ cụ thể

Ba giới hạn dùng chung cho cả bốn họ trong chương. *Dữ liệu huấn luyện* gần như luôn là TartanAir, KITTI hoặc ScanNet, nên phân phối chuyển động và phân phối cảnh đều hẹp hơn hiện trường. *Vấn đề IMU*: dòng đầu-cuối hầu như không dùng IMU, nên nó vứt đi thứ duy nhất còn tin được khi ảnh mờ. *Ngân sách nhúng*: các con số ẩn tượng đều đo trên GPU để bàn, và khoảng cách tới Orin là bậc độ lớn.

Còn một giới hạn ít được nói mà bạn sẽ gặp ngay ngày đầu, đó là ***tính lặp lại***. Bạn đã tốn công truy các nguồn phi tắt định trong hệ hiện tại — thứ tự duyệt container, tranh chấp luồng — để mỗi cấu hình chạy ba lần cho ra số dùng được. Đưa một mạng vào giữa bộ ước lượng là đưa thêm phi tắt định của thư viện GPU vào đúng chỗ đó, và lần này không có bug nào để sửa. Sàn nhiều của harness sẽ dâng lên, và mọi kết luận cũ phải đo lại.

Bảng 1.5. Bốn họ trong chương, đọc theo cùng bốn câu hỏi. Cột cuối là cột quyết định với một hệ stereo-inertial đã tinh chỉnh kỹ, chạy trên phần cứng nhúng.

Họ	Cơ chế cốt lõi	Hỏng khi nào	Lấy riêng được phần nào
Lớp tối ưu khả vi	Đạo hàm qua nghiệm arg min, bằng hàm ẩn hoặc trải phẳng	H suy biến; bộ giải bị cắt ngắn; hoặc không có gì để huấn luyện	Gần như toàn bộ: nó là hạ tầng, không phải một hệ SLAM
Đầu-cuối dày đặc	Dòng quang dày, trọng số học được, BA trong vòng lặp	Ngân sách nhúng; không dùng IMU; bộ nhớ tăng theo độ dài chuỗi	Trọng số liên tục thay quyết định inlier; vòng phản hồi nghiệm về phép đo
Đầu-cuối thưa	Theo vết mảnh ảnh; hiệp phương sai theo từng cặp khớp	Cảnh rất ít vân; vẫn cần GPU; vẫn phụ thuộc phân phối	Hiệp phương sai học được có đơn vị mét — mảnh đáng lấy nhất chương
Tự giám sát và mã ẩn	Mất mát dựng lại ảnh; nén hình học vào mã ngắn	Mơ hồ tỉ lệ; không đóng vòng; tiên nghiệm bịa bề mặt nhất quán	Mẫu “giữ bộ giải cổ điển, học phần dày đặc” kiểu TANDEM

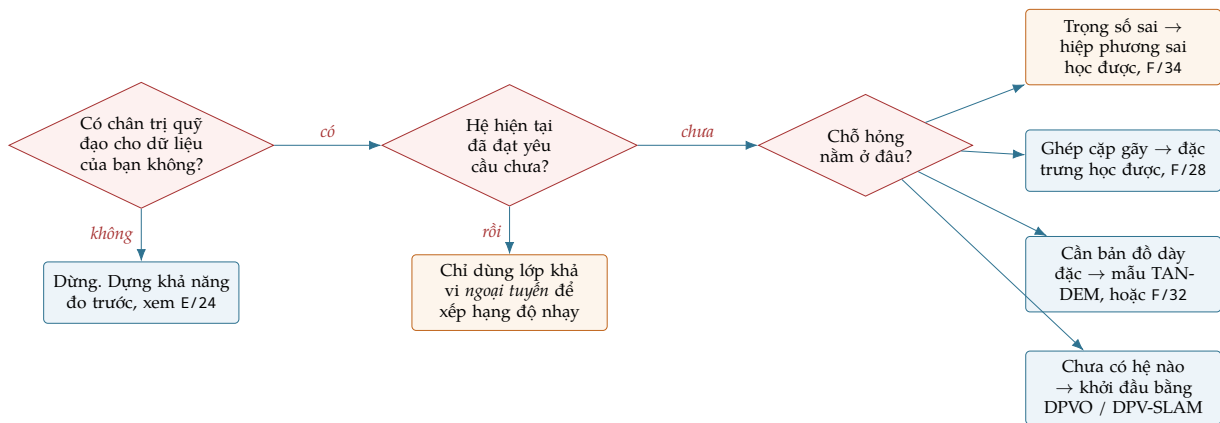
C. Kết luận thẳng thắn

Với một hệ stereo-inertial đã tinh chỉnh kỹ, chạy thời gian thực trên Orin, có harness đo được và có kỷ luật ablation: **hệ đầu-cuối hiện chưa phải lựa chọn thay thế**. Không phải vì ý tưởng sai, mà vì bảng cân đối không có lợi. Bạn sẽ đổi tính chẩn đoán, tính lặp lại và một bậc độ lớn ngân sách tính toán, để lấy một cải thiện chỉ hiện ra trên một phần các cảnh, và mất luôn IMU trên đường đi.

Nhưng kết luận đó không kéo theo “cả chương này vô dụng”. Có hai mảnh tách rời được, và cả hai đều nhỏ hơn nhiều so với việc thay cả hệ.

- **Lớp bất định học được.** Thay ma trận thông tin hằng số $\sigma_\ell^{-2}I$ bằng một hiệp phương sai dự đoán theo từng cặp khớp. Nó cắm vào đúng chỗ backend hiện có đang đọc trọng số, không đụng tới bộ giải, và tắt được bằng một cờ. Đây là mảnh đáng thử trước tiên, và F/34-imu-bat-dinh-va-trien-khai là chỗ bàn nó cho cả camera lẫn IMU.
- **Lớp tối ưu khả vi như một công cụ chẩn đoán ngoại tuyến.** Bạn không cần chạy nó lúc suy luận. Bạn có thể dựng lại bài toán backend của mình trong một lớp khả vi, chạy ngoại tuyến trên dữ liệu đã có chân trị, rồi lấy *gradient* của APE theo các nút vận để biết nút nào thật sự quan trọng. Đó là một công cụ *phân tích độ nhạy*, rẻ hơn quét lưới rất nhiều, và nó không thêm rủi ro nào vào hệ đang chạy.

Hình 1.10 gói cả kết luận đó thành một cây quyết định bốn câu hỏi. Nó cố tình bắt đầu bằng câu hỏi về dữ liệu chứ không phải về phương pháp, vì thiếu chân trị thì mọi nhánh còn lại đều vô nghĩa.



Hình 1.10. Cây quyết định cho cả chương. Câu hỏi đầu tiên cố tình không phải về phương pháp mà về dữ liệu: không có chân trị thì không có *gradient* nào để lấy, nên mọi nhánh phía sau đều vô nghĩa. Hai ô màu cam là hai kết luận “đáng làm” duy nhất cho một hệ stereo-inertial đã tinh chỉnh kỹ; ba ô còn lại chỉ dùng cho các tình huống khác hẳn.

1.8 Thực hành

Repo và tài nguyên. [princeton-vl/lietorch](#) cho phép toán nhóm Lie có *gradient*. [facebookresearch/theseus](#) cho lớp tối ưu phi tuyến khả vi; nên đọc phần tài liệu về các kiểu lượt ngược trước khi đọc mã. [princeton-vl/DRUID-SLAM](#) và [princeton-vl/DPVO](#) để so hai đầu của trục mật độ trên cùng một chuỗi. [nianticlabs/monodepth2](#) là cài đặt sạch nhất của dòng tự giám sát, và là chỗ tốt nhất để tự tay thấy hiện tượng mơ hồ tỉ lệ. [castacks/tartanvo](#) cho ý tưởng tầng nội tham số. *evo* để mọi con số bạn đo có cùng định nghĩa với phần còn lại của dự án. Danh sách này chốt tại tháng 9/2026 và sẽ cũ trong sáu đến mười hai tháng; riêng LieTorch, Theseus và *evo* thuộc loại hạ tầng nên bền hơn hẳn các tên hệ thống đang dẫn đầu bảng.

Câu hỏi kiểm tra hiểu. Viết ra công thức $\partial L / \partial \theta$ qua một lớp arg min mà không nhìn lại, rồi nói rõ mỗi ký hiệu ứng với đại lượng nào trong backend của bạn. Nếu bộ giải chỉ chạy ba bước LM thì đạo hàm theo định lý hàm ẩn sai theo hướng nào, và bạn kiểm tra điều đó bằng thí nghiệm gì? Trong một hệ đầu-cuối, vì sao không có bước nào tương đương với “loại cạnh có χ^2 vượt ngưỡng”?

Bài tập phụ. Lấy một chuỗi EuRoC, chạy DPVO ở chế độ mặc định, rồi vẽ quỹ đạo của nó cạnh quỹ đạo hệ của bạn sau khi căn tỉ lệ. Mục đích không phải so hơn thua. Mục đích là thấy hình dạng lỗi của hai họ khác nhau ra sao: một bên lệch dần và trơn, một bên nhảy tại các khung khó.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Lớp tối ưu khả vi	Một bộ giải Gauss-Newton hoặc LM mà <i>gradient</i> đi ngược xuyên qua được. Lượt xuôi giống hệt bộ giải thường; toàn bộ giá trị nằm ở lượt ngược, và nó chỉ có nghĩa nếu bạn thật sự huấn luyện thứ gì đó bằng nó.
••• Định lý hàm ẩn cho arg min	Tại nghiệm, gradient theo x bằng không; vì phân điều kiện đó cho $\partial x^* / \partial \theta = -H^{-1} B$. Nhờ nó, lượt ngược chỉ tốn thêm một lần giải hệ với chính H đã phân tích ở lượt xuôi.
••• Trái phải (unrolling)	Coi K bước lặp như một mạng K tầng rồi để autodiff chạy. Đạo hàm đúng với thuật toán đã chạy, kể cả khi chưa hội tụ, nhưng bộ nhớ tăng tuyến tính theo K .

Thuật ngữ / câu hỏi	Trả lời (tiếp)
•• Khi nào định lý hàm ẩn cho đạo hàm sai?	Khi bước cuối không phải điểm dừng thật, tức bộ giải bị cắt ngắn. Hoặc khi H suy biến vì tự do đo chưa được neo. Cả hai đều thường xuyên xảy ra trong SLAM.
••• Vì sao hệ đầu-cuối không xuất mặt nạ inlier?	Vì so sánh với ngưỡng là hàm bậc thang, đạo hàm bằng 0 hầu khắp nơi. Muốn học được thì phải thay quyết định cứng bằng một trọng số liên tục.
•• LieTorch	Thư viện lan truyền <i>gradient</i> trong không gian tiếp tuyến của nhóm Lie. Nó làm số chiều <i>gradient</i> bằng đúng số bậc tự do, sáu cho $SE(3)$, nên bỏ được việc chuẩn hoá quaternion giữa đồ thị.
•• Theseus	Lớp tối ưu phi tuyến khả vi trong PyTorch, có bộ giải thừa. Điểm đáng học là nó bắt người dùng chọn kiểu lượt ngược, tức thừa nhận thẳng rằng đạo hàm qua bộ giải là một xấp xỉ có thiên lệch.
•• BA-Net	Đặt cả một lớp hiệu chỉnh chùm tia vào mạng. Ba ý: sai lệch trên đặc trưng thay vì trên độ sáng, damping học được, và độ sâu theo cơ sở. Ý đầu bền nhất vì căn chỉnh trên đặc trưng học được có lòng chảo hội tụ rộng hơn và bất biến với đổi sáng.
••• DROID-SLAM	Toán tử hồi quy kiểu RAFT sinh dòng quang dày và trọng số theo từng điểm ảnh; một lớp BA dày đặc giải tư thế và độ sâu; nghiệm quay lại làm đầu vào cho vòng sau, nhờ đó hệ tự gỡ được khởi tạo tồi. Chính xác, nhưng cần GPU để bàn và không dùng IMU.
••• DPVO	Thưa hoá DROID bằng cách theo vết khoảng một trăm mảnh ảnh mỗi khung, mỗi mảnh một ẩn độ sâu. Số ẩn hình học giảm vài bậc độ lớn, đủ để hệ chạy trên một GPU để bàn duy nhất thay vì nhiều card.
•• DPV-SLAM	DPVO cộng thêm đóng vòng và tối ưu đồ thị tư thế. Bài học lớn hơn kết quả: đóng vòng và tối ưu đồ thị tư thế vẫn là hai khối cổ điển mà chưa ai học thay được.
••• MAC-VO	Dự đoán hiệp phương sai có đơn vị mét cho từng cặp khớp stereo, dùng để chọn điểm và để làm ma trận thông tin. Đây là mảnh ghép nhất với thứ một hệ cổ điển mượn được ngay.
••• SfM-Learner	Huấn luyện mạng độ sâu và mạng tư thế chỉ bằng video thô, tín hiệu dạy là mất mát dựng lại ảnh. Cần một mặt nạ giải thích được, và mặt nạ đó phải bị chính quy hoá vì nghiệm suy biến là tất hết.
••• Mơ hồ tỉ lệ với ảnh đơn	Nhân độ sâu và tịnh tiến với cùng một hằng số cho ra đúng cùng ảnh dựng lại, nên mất mát không phân biệt được. Hệ quả: nhiều bảng kết quả căn tỉ lệ theo từng chuỗi, việc mà robot không làm được.
•• DeepVO tốt trên KITTI mà kém chỗ khác — nguyên nhân?	Nó học luôn tiên nghiệm chuyển động của ô tô: đi thẳng, tốc độ trong dải hẹp, góc lệch nhỏ. Trên camera cầm tay hoặc drone, tiên nghiệm ấy sai theo hướng có hệ thống.
•• TartanVO	Chuẩn hoá tịnh tiến để bỏ tỉ lệ ra khỏi hàm mất mát, và nổi nội tham số camera vào đầu vào. Nhờ hai thủ thuật đó, một mô hình chạy được qua nhiều tập dữ liệu mà không tinh chỉnh lại.
•• TANDEM	Giữ tracking trực tiếp cổ điển, chỉ học phần dựng bản đồ dày đặc rồi hợp nhất TSDF. Đây là mẫu thiết kế đáng dùng nhất trong dòng tự giám sát cho một hệ đã chạy tốt.
••• CodeSLAM	Nén bản đồ độ sâu vào một mã cỡ vài chục chiều, với bộ giải mã gần tuyến tính theo mã, để đưa mã vào đồ thị nhân tử cạnh tư thế. Tính gần tuyến tính là điều kiện sống còn, nếu không thì mỗi lần tuyến tính hoá tốn một lượt qua mạng.
••• Vì sao dòng mã ngắn chưa thẳng?	Sai số của tiên nghiệm trơn và nhất quán giữa các khung nên cổng χ^2 không bắt được. Thêm nữa bán kính hiệu lực của phép tuyến tính hoá nhỏ, độ phủ hẹp, và 3DGS đã lấy mất phần thường “bản đồ dày đặc”.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
● ● ● Ngoại lai thừa và độc lập	Giả định nền của mọi ước lượng robust. Tiên nghiệm học được vi phạm nó, vì cùng một bộ giải mã sinh ra sai số ở mọi khung nên chúng đồng ý với nhau và trông như hình học thật.

● ● ● phải thuộc ● ● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
LieTorch [5]	Lan truyền <i>gradient</i> trong không gian tiếp tuyến của nhóm Lie — hạ tầng cho gần như mọi hệ trong chương	Hạ tầng; bền hơn tên hệ thống
Theseus [7]	Lớp Gauss-Newton và LM khả vi, bắt người dùng khai báo kiểu lượt ngược: trải phẳng, cắt ngắn, hay hàm ẩn	Hạ tầng; dùng được ngoại tuyến để phân tích độ nhạy
DROID-SLAM [10] và các nhánh dẫn xuất	BA dày đặc trong vòng lặp, cộng trọng số học được theo từng điểm ảnh	Vẫn là mốc chính xác tham chiếu cho nhiều bảng SLAM học sâu
DPVO và DPV-SLAM [13], [14]	Thừa hoá bằng mảnh ảnh; và việc phải mượn lại đóng vòng cổ điển	Ví dụ rõ nhất rằng đóng vòng chưa được học thay
MAC-VO [15]	Hiệp phương sai học được theo từng cặp khớp, dùng làm ma trận thông tin	Sau 2024; mình nêu ý tưởng, chưa tự kiểm chứng số liệu
Dòng monodepth2 và các hệ độ sâu tự giám sát	Mất mát dừng lại ảnh và mặt nạ giải thích được của SfM-Learner [16]	Ý tưởng còn sống mạnh ở phía độ sâu hơn là ở phía tư thế

MINI PROJECT SLAM · Đo độ nhạy của APE theo trọng số cạnh, bằng đạo hàm thay vì quét lưới · 1–2 ngày

Mục tiêu	biến câu “trọng số cạnh đặt bằng tay” thành một bảng số nói rõ nút vặn nào thật sự lay chuyển quỹ đạo. Đây là phiên bản rẻ nhất và không rủi ro của toàn bộ ý tưởng khả vi: bạn chỉ dùng <i>gradient</i> để <i>chẩn đoán</i> , không đưa mạng nào vào hệ đang chạy.
Dữ liệu	một chuỗi EuRoC có chân trị. Lấy đúng một cửa sổ 8–10 khung then chốt từ log của hệ hiện tại, kèm toàn bộ quan sát và khởi tạo của cửa sổ đó.
Các bước	dựng lại đúng bài toán bình phương tối thiểu của cửa sổ ấy trong một lớp tối ưu khả vi; kiểm tra nghiệm khớp với nghiệm của backend C++ tới cỡ 10^{-4} trước khi làm bất cứ việc gì khác; đặt L là sai lệch tư thế so với chân trị; lấy $\partial L / \partial \theta$ cho ba nhóm tham số là trọng số cạnh chiếu lại theo tầng kim tự tháp, bề rộng nhân robust, và trọng số cạnh IMU; cuối cùng đối chiếu với đạo hàm số bằng sai phân hữu hạn trên vài thành phần.
Xong khi	bạn có một bảng xếp hạng độ nhạy, và kiểm chứng được nó bằng một thí nghiệm độc lập. Đổi tay đúng nút vặn đúng đầu bảng một lượng nhỏ, rồi xác nhận APE đổi theo đúng dấu mà <i>gradient</i> dự đoán, trên ít nhất ba lần chạy để vượt sàn nhiễu. Nếu dấu không khớp thì gần như chắc chắn cửa sổ của bạn chưa hội tụ tới điểm dừng. Đó cũng là một kết quả, và nó nói rằng định lý hàm ẩn chưa dùng được ở đây.
Nối với	E/24-thi-nghiem-va-ablation cho sàn nhiễu và cỡ mẫu; F/34-imu-bat-dinh-va-trien-khai sẽ dùng lại chính bảng độ nhạy này để quyết định có đáng học hiệp phương sai hay không; A/01-toan-toi-thieu/02-vi-phan-va-toi-uu cho phần kiểm tra đạo hàm bằng sai phân hữu hạn.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Nó không làm Gauss-Newton hội tụ nhanh hơn, vì lượt xuôi y hết. Nó bán một thứ khác: đạo hàm của lỗi quỹ đạo theo các nút vận mà bạn đang đặt bằng tay, tức là biến việc quét lưới thành việc huấn luyện.
- A2. Vì tại nghiệm, gradient theo x bằng không, và điều kiện đó ràng buộc nghiệm với tham số. Vì phân nó ra $-H^{-1}B$, chỉ cần H ở bước cuối. Điều kiện là H phải khả nghịch và bước cuối phải là điểm dừng thật; trong SLAM cả hai đều hay bị vi phạm, vì tự do đo và vì bộ giải bị cắt ngắn.
- A3. Không học được, nên các hệ đầu-cuối không cố học. Chúng thay quyết định cứng bằng một trọng số liên tục cho từng cặp khớp: mạng hạ trọng số điểm xấu thay vì loại nó. Đây là hệ quả bắt buộc của việc muốn có *gradient*, không phải một lựa chọn thẩm mỹ.
- A4. Thứ chặn nó là ngân sách tính toán và bộ nhớ. Nó giữ một ẩn độ sâu cho *mỗi* điểm ảnh ở $1/8$ độ phân giải, cộng các khối tương quan có kích thước bằng bình phương số vị trí cho mỗi cạnh của đồ thị khung. Hệ được công bố trên GPU để bàn cao cấp, còn máy nhúng thấp hơn hẳn về cả tính toán lẫn băng thông — khoảng cách thuộc loại bậc độ lớn chứ không phải vài chục phần trăm. Thêm nữa nó không dùng IMU.
- A5. Hồng ở chỗ sai số của bộ giải mã trơn và nhất quán giữa các khung, nên nó vượt qua mọi cổng robust vốn giả định ngoại lai thừa và độc lập. Cộng thêm ba điều: phép tuyến tính hoá chỉ đúng quanh mã ban đầu, độ phủ hẹp ngoài phân phối huấn luyện, và 3DGS đã cho bản đồ dày đặc tốt hơn mà không cần tiên nghiệm hình dạng.

NÓI LẠI THẬT ĐƠN GIẢN

Hình dung bạn có một hội đồng gồm hai trăm người cùng đoán một con số. Bạn không lấy trung bình tròn. Bạn lấy trung bình có trọng số: ai bạn tin hơn thì tiếng nói nặng hơn. Từ trước tới nay bạn phát trọng số theo một quy tắc cứng, kiểu “ai ngồi gần bảng thì tin hơn”. Quy tắc đó không quan tâm hôm nay ai quên kính, ai ngồi ngược sáng, ai vừa ngủ dậy.

Bây giờ giả sử cuối buổi bạn biết đáp án đúng. Câu hỏi tự nhiên là: nếu cho người thứ 47 nói to hơn một chút, kết quả chung sẽ dịch về phía đáp án hay ra xa nó? Trả lời được câu đó cho cả hai trăm người thì bạn có một cách phát trọng số tốt hơn mọi quy tắc cứng. Điều bất ngờ là bạn không cần diễn lại cả buổi họp để trả lời. Chỉ cần biết kết quả chung nhảy thể nào quanh điểm cân bằng của nó, rồi làm một phép tính.

Cái giá thì có ba khoản. Muốn học kiểu đó, bạn phải có nhiều buổi họp đã biết đáp án. Phép tính trên trở nên rất nhạy đúng vào những buổi mà hội đồng chia rẽ nhất, mà đó lại là những buổi bạn cần học nhất. Và khi trọng số do một cỗ máy phát ra, hôm nào kết quả sai bạn sẽ không còn chỉ được vào ai đã nói sai. Bạn chỉ biết rằng nó sai.

Chỗ không rút gọn được: chỗ vấp thật nằm ở các quyết định bật-tắt. Nhiều thứ trong một hệ SLAM không phải chuyện tin nhiều hay tin ít, mà là chuyện có mời người đó vào phòng họp hay không: chèn khung then chốt hay không, nhận một vòng lặp hay không. Với những quyết định đó, câu hỏi “nói to hơn một chút thì sao” không có nghĩa, vì không có “một chút” nào cả. Mọi cách nói đơn giản mà mình biết đều làm mờ mất ranh giới này, trong khi chính nó mới là ranh giới thật giữa phần học được và phần không học được.

Một câu: *Khả vi không làm bộ giải khôn hơn; nó chỉ cho phép hỏi “tin phép đo này thêm một chút thì quỹ đạo tốt lên hay xấu đi”, và bạn trả tiền cho câu hỏi đó bằng bộ nhớ, độ ổn định và khả năng chắn đoán.*

1.9 Kết nối

F/27-dat-hoc-sau-vao-dau là bộ lọc mà chương này vừa được đưa qua: bốn cổng của nó cho ra kết luận “chưa đáng” với cả hệ, và “đáng thử” với hai mảnh nhỏ. F/28-dac-trung-va-ghep-cap là nửa còn lại của cùng một câu chuyện — ở đó phần học được nằm trước bộ giải, ở đây nó nằm bên trong. F/30-do-sau-dong-quang-va-mo-hinh-nen là tiền đề vật liệu: dòng quang và độ sâu học được chính là đầu vào mà mọi hệ trong chương này tiêu thụ. F/32-ban-do-neural là nơi cuộc thi về biểu diễn bản đồ dày đặc chuyển sang, và là lý do dòng mã ngăn mất động lực.

F/34-imu-bat-dinh-va-trien-khai là chỗ mảnh đáng lấy nhất của chương, tức hiệp phương sai học được, được bàn cho đủ, cùng với ngân sách độ trễ trên Orin.

Ngoài phần F thì có bốn mối nối. A/01/02 là nền của định lý hàm ẩn và của cả hai lối đi ngược. C/15/03 giải thích toán tử cập nhật hồi quy mà DROID-SLAM mượn lại. B/11-gioi-han-tinh-toan cho khung để biến các con số ẩn số và tensor ở mục 4 thành một ngân sách bộ nhớ thật. E/24 là kỷ luật bắt buộc nếu bạn định thử một trong hai mảnh tách rời: đo sần nhiều mới trước, nói về cải tiến sau.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao đạo hàm theo định lý hàm ẩn chỉ tốn thêm một lần thế ngược, trong khi trải phẳng tốn bộ nhớ tỉ lệ với số bước lặp?
2. Bộ giải của bạn dừng sau ba bước LM vì ngân sách thời gian. Đạo hàm thu được khi đó là đạo hàm của cái gì, và bạn thiết kế thí nghiệm nào để phát hiện sai lệch ấy?
3. Nếu quyết định inlier có đạo hàm bằng 0 hầu khắp nơi, thì cơ chế nào trong DROID-SLAM đang đóng vai trò loại điểm sai, và nó khác gì về hành vi khi gặp một cảnh ngoài phân phối huấn luyện?
4. Vì sao *gradient* lại lớn và nhiều nhất đúng ở các khung thị sai thấp hoặc quay thuần túy, và vì sao mọi cách chữa đều làm thiên lệch việc học ở chính các khung đó?
5. Một hệ VO tự giám sát báo lỗi quỹ đạo rất nhỏ trên KITTI. Bạn phải kiểm đúng một điều trước khi tin con số đó — điều gì, và vì sao nó quyết định con số ấy có dùng được cho robot hay không?
6. Vì sao một bề mặt do bộ giải mã bịa ra lại nguy hiểm hơn một cặp khớp sai, xét theo giả định mà nhân robust dựa vào?
7. Nếu chỉ được lấy đúng một mảnh từ chương này về hệ hiện tại, bạn lấy mảnh nào, và bạn đo cái gì để biết nó có tác dụng vượt sần nhiều?

Nguồn

- [1] C. Campos, R. Elvira, J. J. G. Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM,” *IEEE T-RO*, vol. 37, no. 6, pp. 1874–1890, 2021.
- [2] B. Amos and J. Z. Kolter, “OptNet: Differentiable Optimization as a Layer in Neural Networks,” in *International Conference on Machine Learning (ICML)*, 2017.
- [3] A. Agrawal, B. Amos, S. Barratt, et al., “Differentiable Convex Optimization Layers,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [4] S. Bai, J. Z. Kolter, and V. Koltun, “Deep Equilibrium Models,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [5] Z. Teed and J. Deng, “Tangent Space Backpropagation for 3D Transformation Groups,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [6] T. D. Barfoot, *State Estimation for Robotics*. Cambridge University Press, 2017.
- [7] L. Pineda, T. Fan, M. Monge, et al., “Theseus: A Library for Differentiable Nonlinear Optimization,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [8] K. M. Jatavallabhula, G. Iyer, and L. Paull, “gradSLAM: Dense SLAM meets Automatic Differentiation,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2020.
- [9] C. Tang and P. Tan, “BA-Net: Dense Bundle Adjustment Networks,” in *International Conference on Learning Representations (ICLR)*, 2019.
- [10] Z. Teed and J. Deng, “DROID-SLAM: Deep Visual SLAM for Monocular, Stereo, and RGB-D Cameras,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.
- [11] Z. Teed and J. Deng, “RAFT: Recurrent All-Pairs Field Transforms for Optical Flow,” in *ECCV*, 2020.
- [12] W. Wang, D. Zhu, X. Wang, et al., “TartanAir: A Dataset to Push the Limits of Visual SLAM,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2020.
- [13] Z. Teed, L. Lipson, and J. Deng, “Deep Patch Visual Odometry,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [14] L. Lipson, Z. Teed, and J. Deng, “Deep Patch Visual SLAM,” in *European Conference on Computer Vision (ECCV)*, 2024.
- [15] Y. Qiu, Y. Chen, Z. Zhang, et al., “MAC-VO: Metrics-aware Covariance for Learning-based Stereo Visual Odometry,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2025.
- [16] T. Zhou, M. Brown, N. Snavely, et al., “Unsupervised Learning of Depth and Ego-Motion from Video,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [17] R. Li, S. Wang, Z. Long, et al., “UnDeepVO: Monocular Visual Odometry through Unsupervised Deep Learning,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2018.
- [18] S. Wang, R. Clark, H. Wen, et al., “DeepVO: Towards End-to-End Visual Odometry with Deep Recurrent Convolutional Neural Networks,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2017.

- [19] W. Wang, Y. Hu, and S. Scherer, "TartanVO: A Generalizable Learning-based VO," in *Conference on Robot Learning (CoRL)*, 2020.
- [20] H. Zhou, B. Ummenhofer, and T. Brox, "DeepTAM: Deep Tracking and Mapping," in *European Conference on Computer Vision (ECCV)*, 2018.
- [21] Z. Teed and J. Deng, "DeepV2D: Video to Depth with Differentiable Structure from Motion," in *International Conference on Learning Representations (ICLR)*, 2020.
- [22] F. Wimbauer, N. Yang, L. von Stumberg, et al., "MonoRec: Semi-Supervised Dense Reconstruction in Dynamic Environments from a Single Moving Camera," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [23] L. Koestler, N. Yang, N. Zeller, et al., "TANDEM: Tracking and Dense Mapping in Real-time using Deep Multi-view Stereo," in *Conference on Robot Learning (CoRL)*, 2021.
- [24] M. Bloesch, J. Czarnowski, R. Clark, et al., "CodeSLAM: Learning a Compact, Optimisable Representation for Dense Visual SLAM," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.
- [25] S. Zhi, M. Bloesch, S. Leutenegger, et al., "SceneCode: Monocular Dense Semantic Reconstruction using Learned Encoded Scene Representations," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [26] J. Czarnowski, T. Laidlow, R. Clark, et al., "DeepFactors: Real-Time Probabilistic Dense Monocular SLAM," *IEEE Robotics and Automation Letters*, 2020.
- [27] E. Sucar, K. Wada, and A. J. Davison, "NodeSLAM: Neural Object Descriptors for Multi-View Shape Reconstruction," in *International Conference on 3D Vision (3DV)*, 2020.
- [28] H. Matsuki, R. Scona, J. Czarnowski, et al., "CodeMapping: Real-Time Dense Mapping for Sparse SLAM using Compact Scene Representations," *IEEE Robotics and Automation Letters*, 2021.
- [29] B. Kerbl, G. Kopanas, T. Leimkühler, et al., "3D Gaussian Splatting for Real-Time Radiance Field Rendering," *ACM TOG*, vol. 42, no. 4, 2023.